

Implementing the ISA: Basics

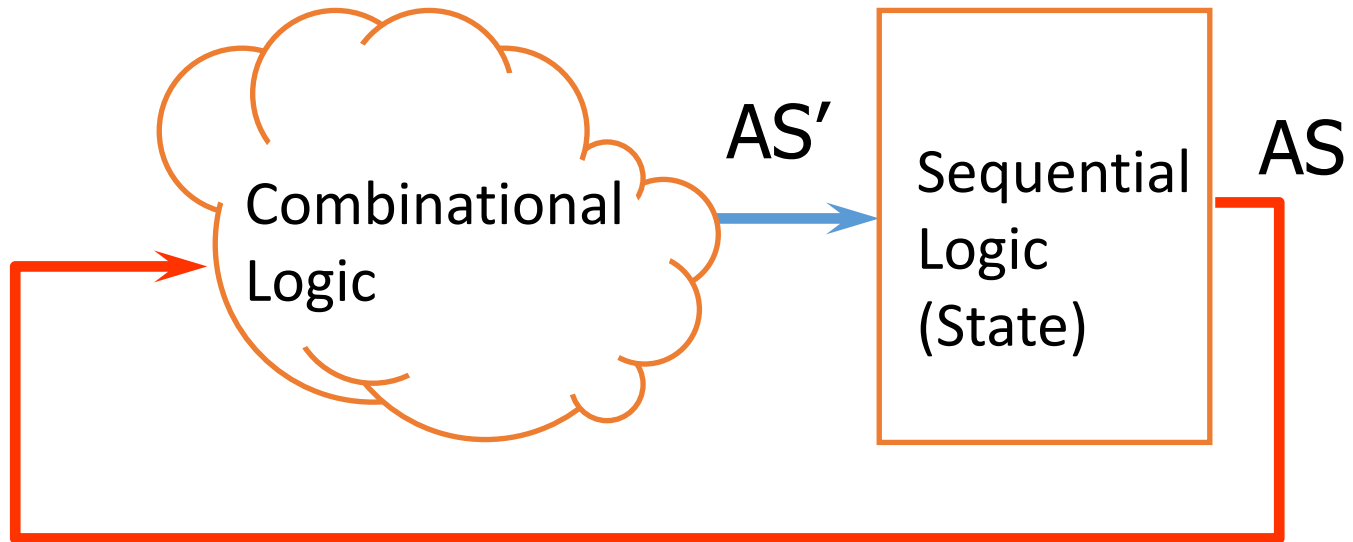
Microarchitecture

A Single-Cycle Microarchitecture

A Closer Look

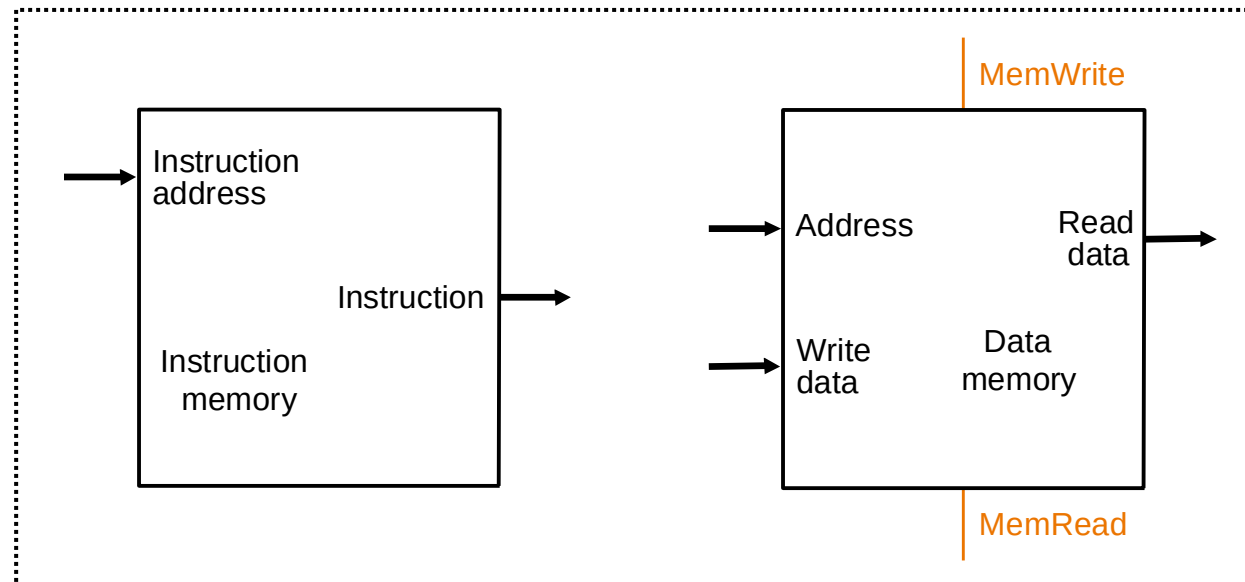
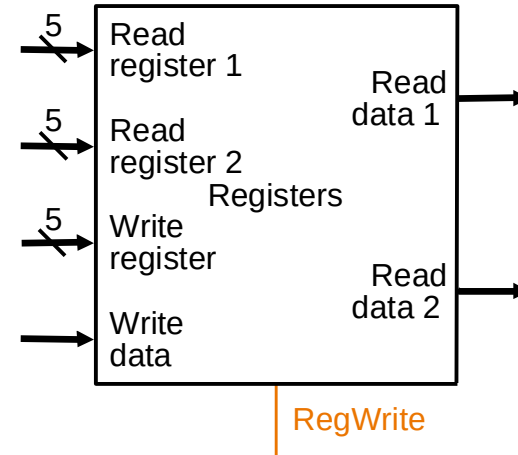
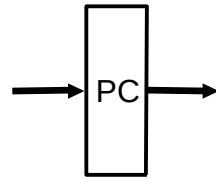
Remember...

- Single-cycle machine

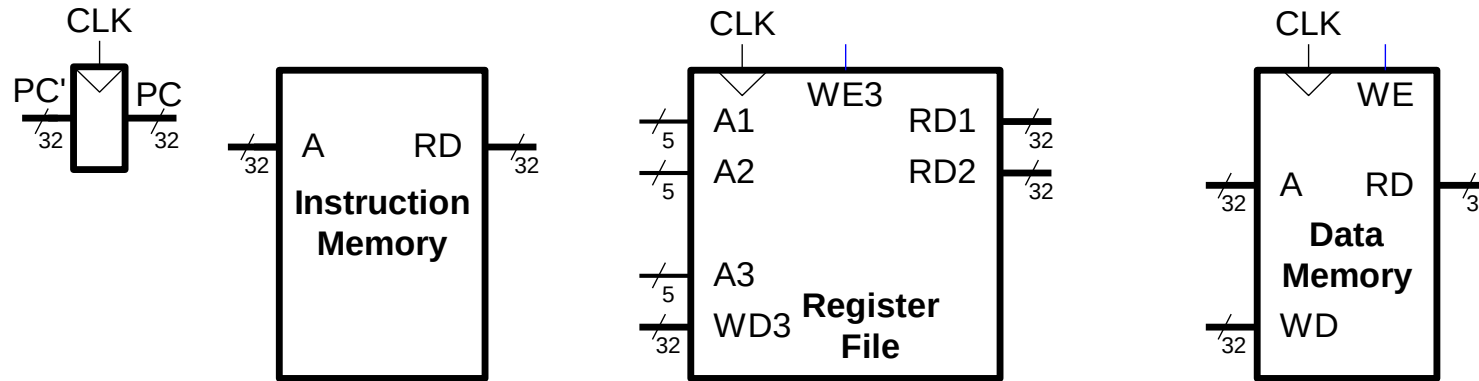


Let's Start with the State Elements

- Data and control inputs



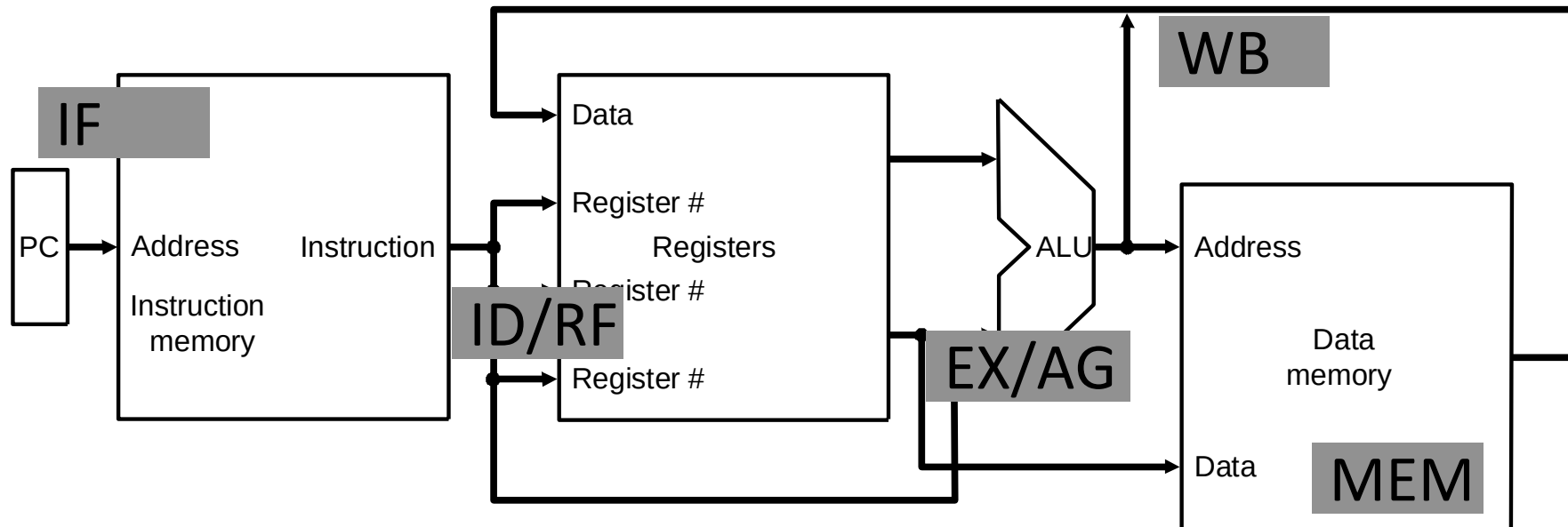
MIPS State Elements



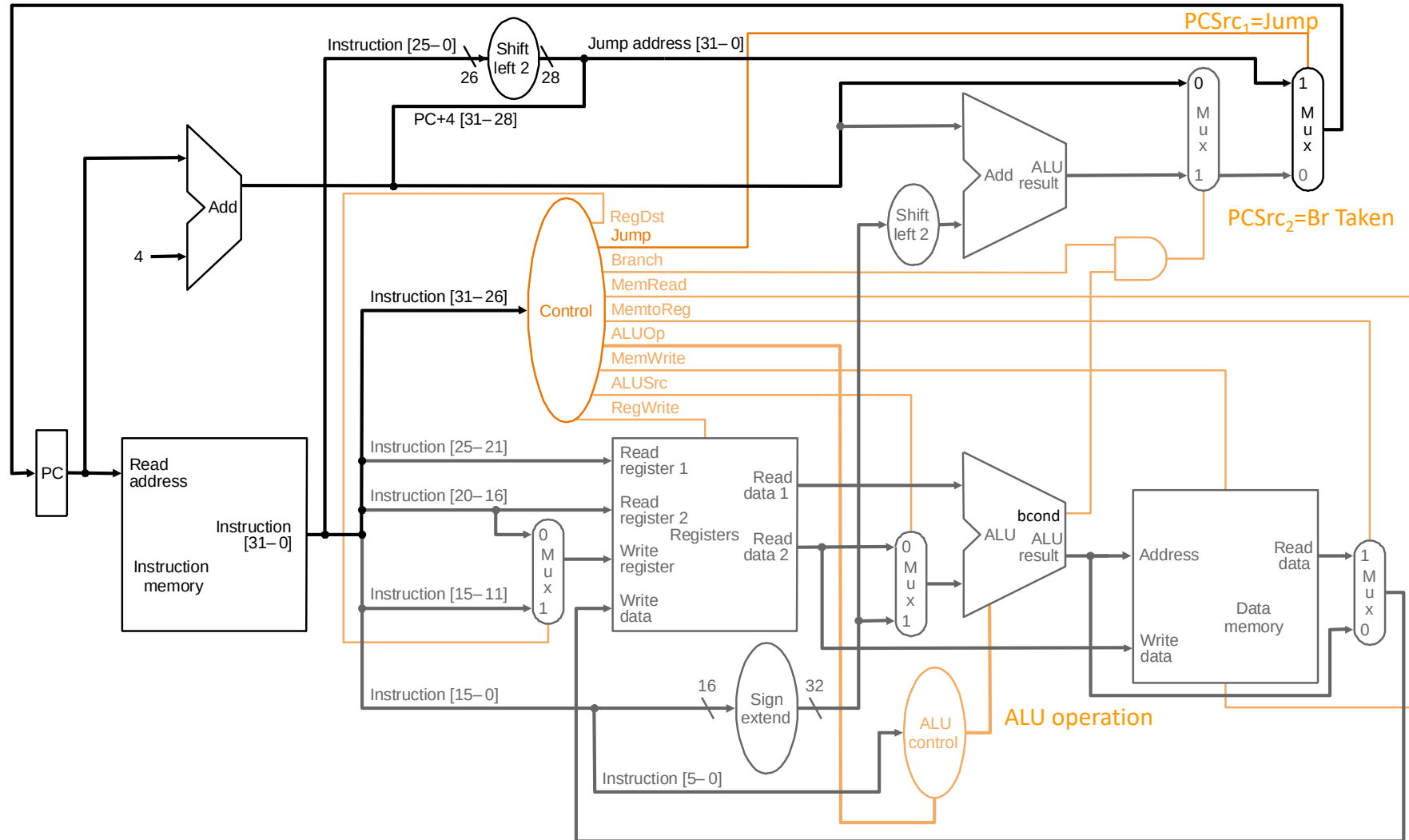
- **Program counter:**
32-bit register
- **Instruction memory:**
Takes input 32-bit address A and reads the 32-bit data (i.e., instruction) from that address to the read data output RD.
- **Register file:**
The 32-element, 32-bit register file has 2 read ports and 1 write port
- **Data memory:**
Has a single read/write port. If the write enable, WE, is 1, it writes data WD into address A on the rising edge of the clock. If the write enable is 0, it reads address A onto RD.

Instruction Processing

- 5 generic steps
 - Instruction fetch (IF)
 - Instruction decode and register operand fetch (ID/RF)
 - Execute/Evaluate memory address (EX/AG)
 - Memory operand fetch (MEM)
 - Store/writeback result (WB)

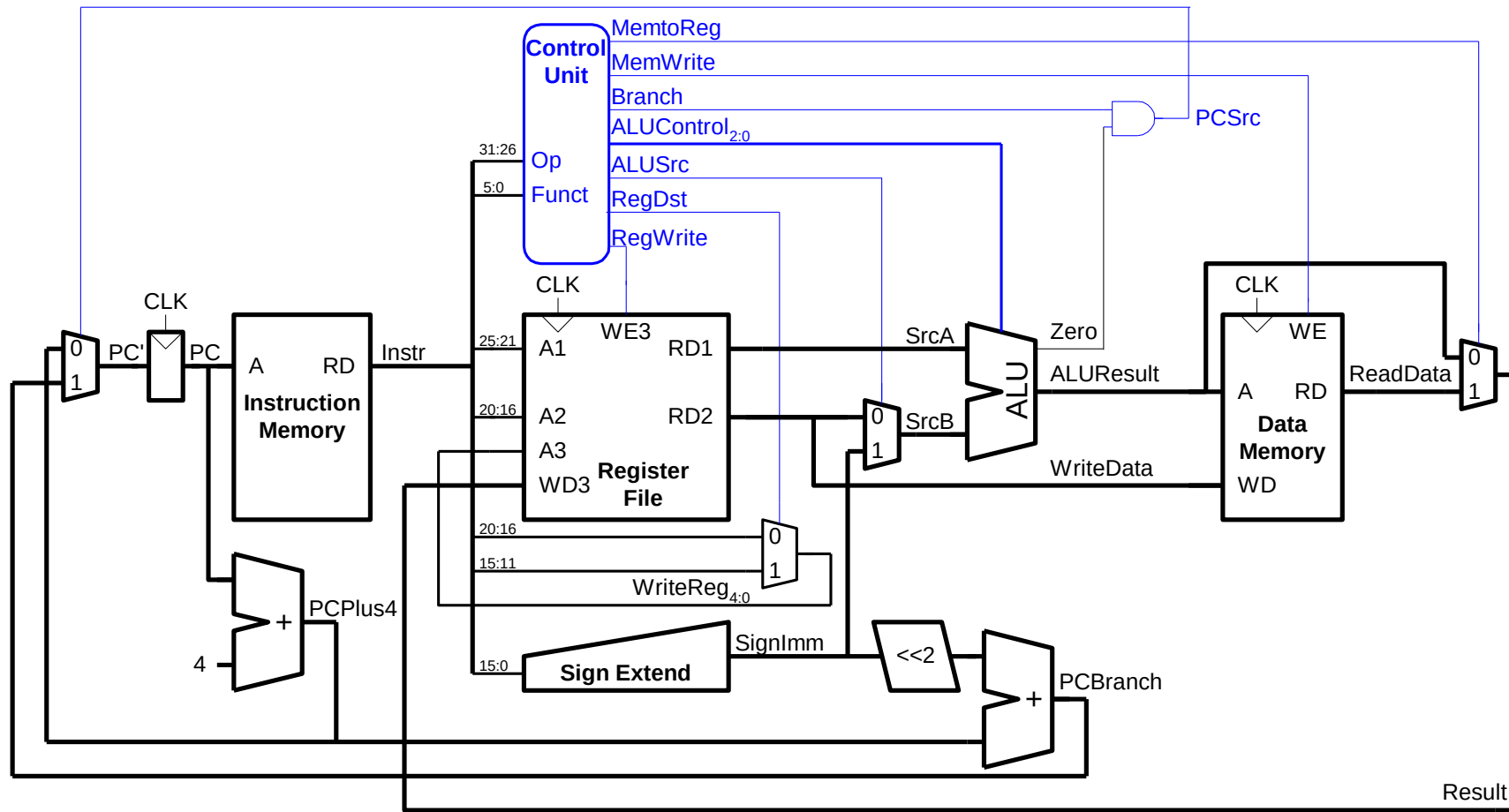


What Is To Come: The Full MIPS Datapath



JAL, JR, JALR omitted

Another Complete Single-Cycle Processor



Single-Cycle Datapath for *Arithmetic and Logical Instructions*

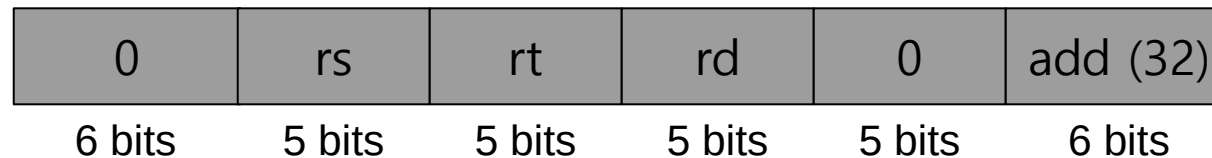
R-Type ALU Instructions

- R-type: 3 register operands

MIPS assembly (e.g., register-register signed addition)

```
add    $s0, $s1, $s2    # $s0=rd, $s1=rs, $s2=rt
```

Machine Encoding

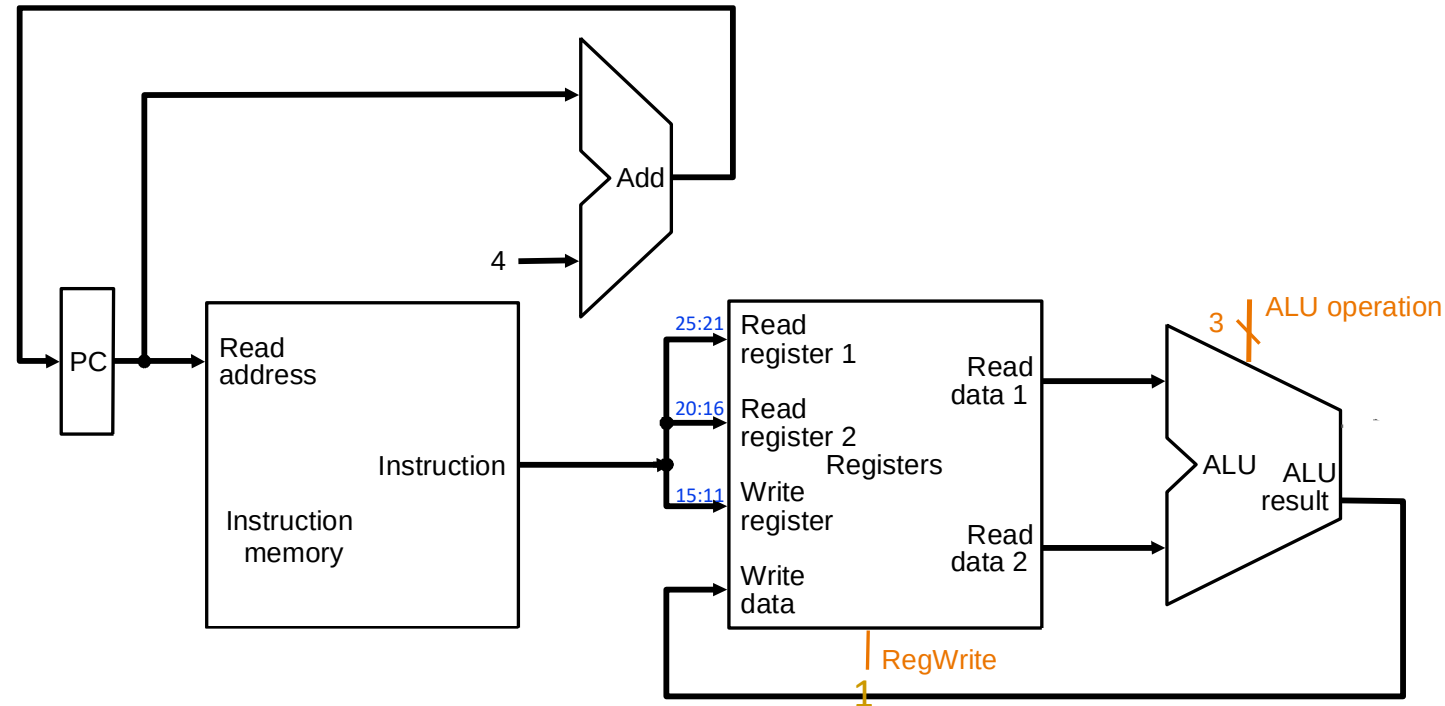


R-Type

- Semantics

```
if MEM[PC] == add rd rs rt
    GPR[rd] ← GPR[rs] + GPR[rt]
    PC ← PC + 4
```

(R-Type) ALU Datapath



if MEM[PC] == ADD rd rs rt
GPR[rd] $\leftarrow$ GPR[rs] + GPR[rt]
PC $\leftarrow$ PC + 4

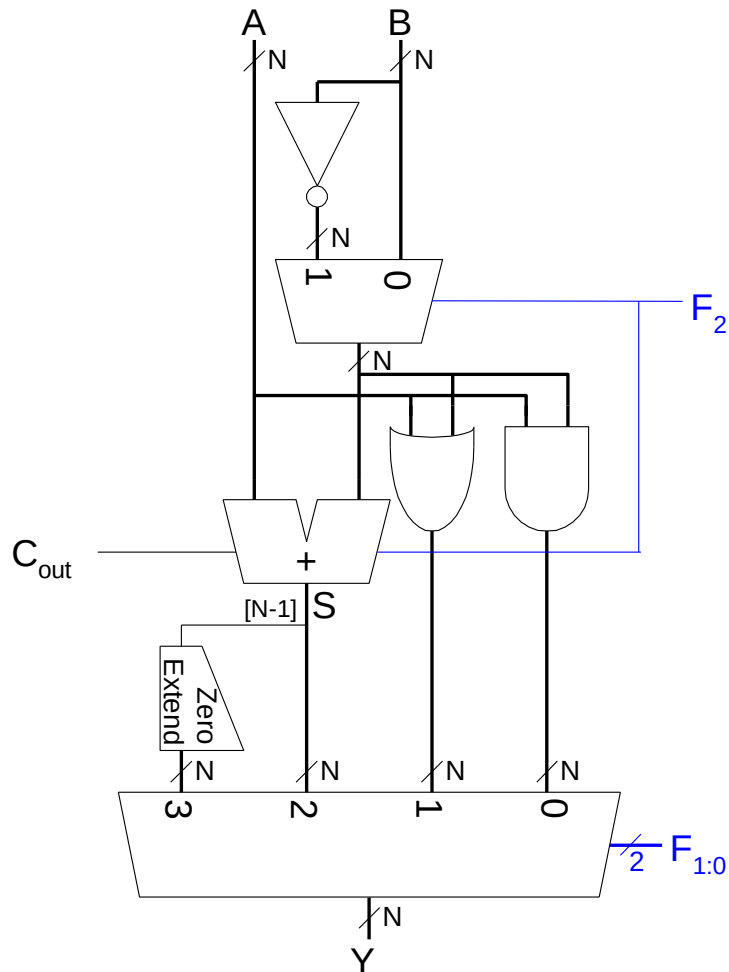


IF	ID	EX	MEM	WB
----	----	----	-----	----

Combinational
state update logic

Example: ALU Design

- **ALU operation** ($F_{2:0}$) comes from the control logic



$F_{2:0}$	Function
000	A & B
001	A B
010	A + B
011	not used
100	A & $\sim$ B
101	A $\sim$ B
110	A - B
111	SLT

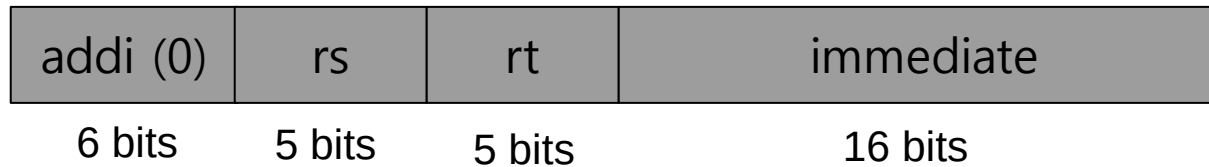
I-Type ALU Instructions

- I-type: 2 register operands and 1 immediate

MIPS assembly (e.g., register-immediate signed addition)

```
addi $s0, $s1, 5    # $s0 = rt, $s1 = rs
```

Machine Encoding



I-Type

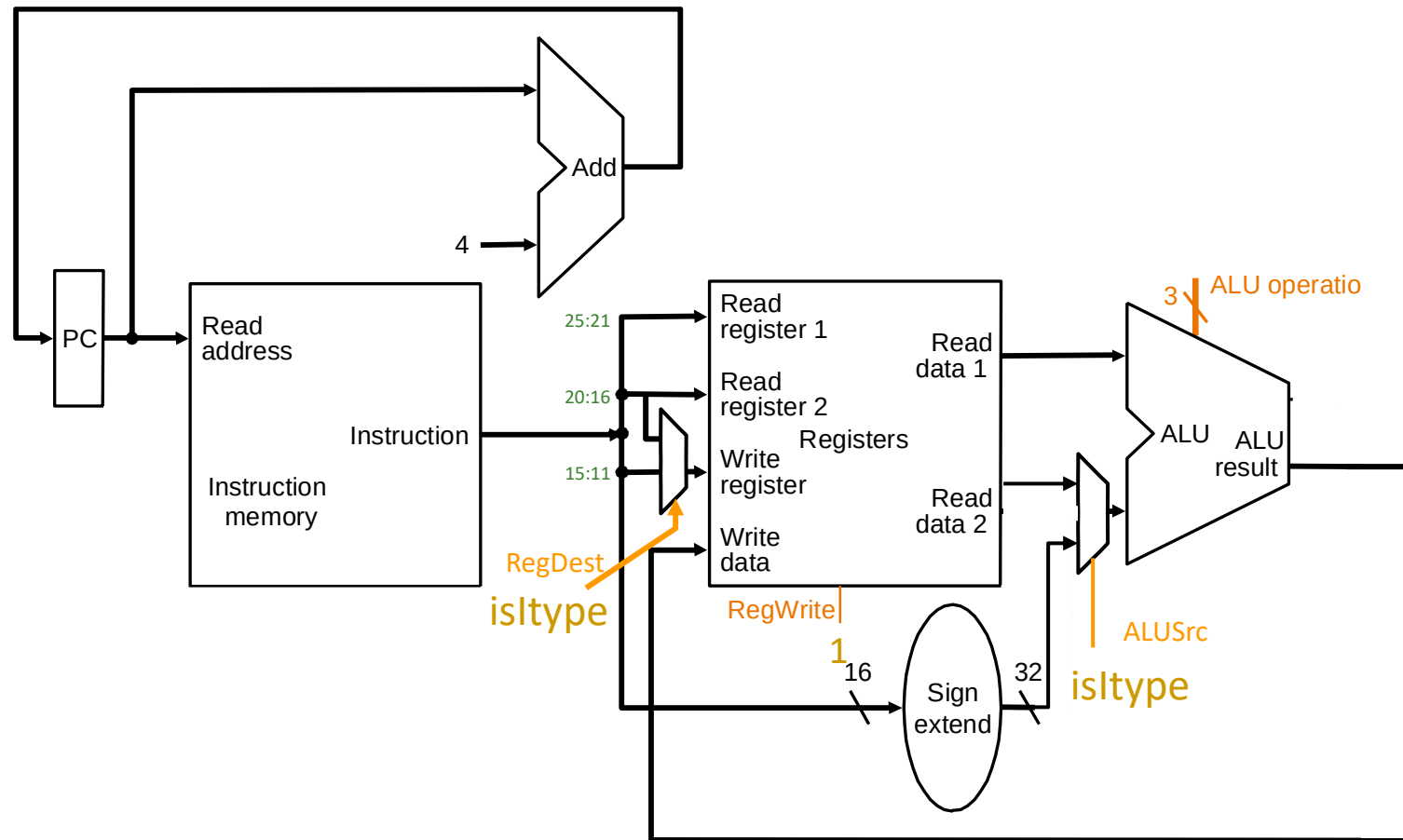
- Semantics

if MEM[PC] == addi rs rt immediate

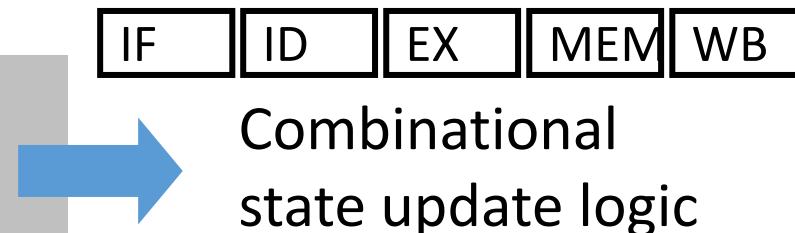
PC $\leftarrow$ PC + 4

GPR[rt] $\leftarrow$ GPR[rs] + sign-extend(immediate)

Datapath for R and I-Type ALU Insts.



```
if MEM[PC] == ADDI rt rs immediate
    GPR[rt] ← GPR[rs] + sign-extend(immediate)
    PC ← PC + 4
```



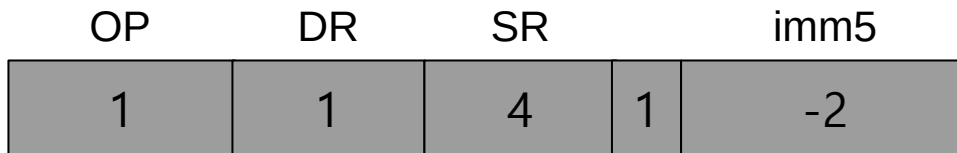
Recall: ADD with one Literal in LC-3

- ADD assembly and machine code

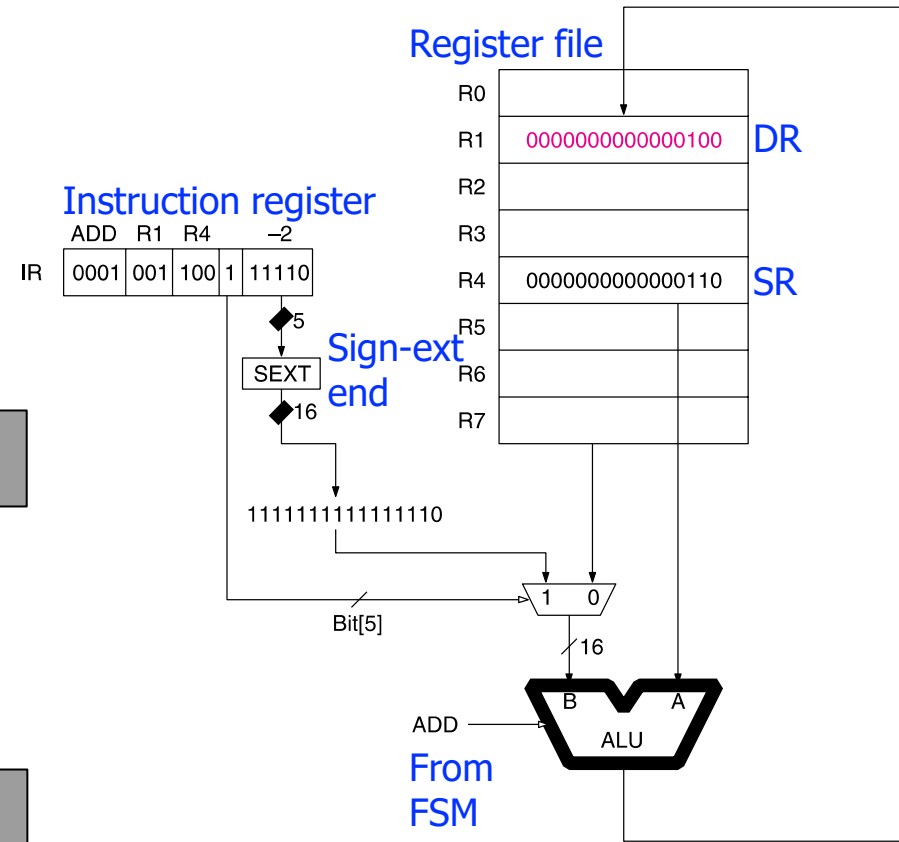
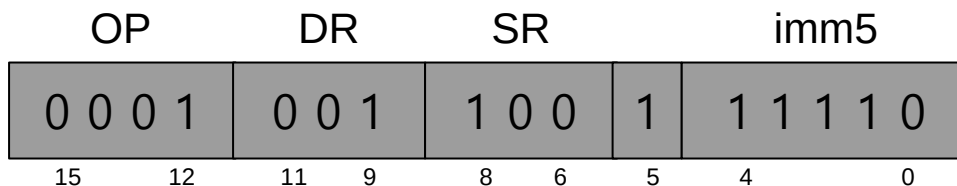
LC-3 assembly

```
ADD R1, R4, #-2
```

Field Values



Machine Code



Single-Cycle Datapath for *Data Movement Instructions*

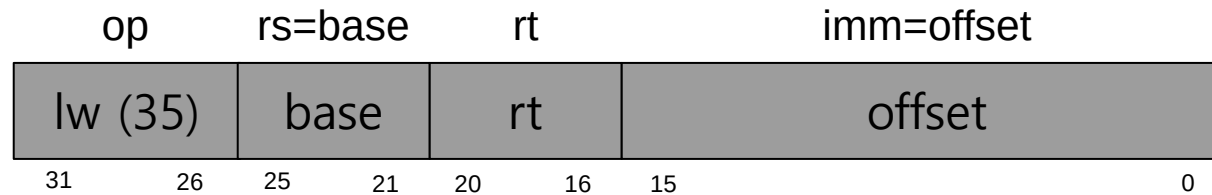
Load Instructions

- Load 4-byte word

MIPS assembly

```
lw    $s3, 8($s0)    # $s0=rs, $s3=rt
```

Machine Encoding



I-Type

- Semantics

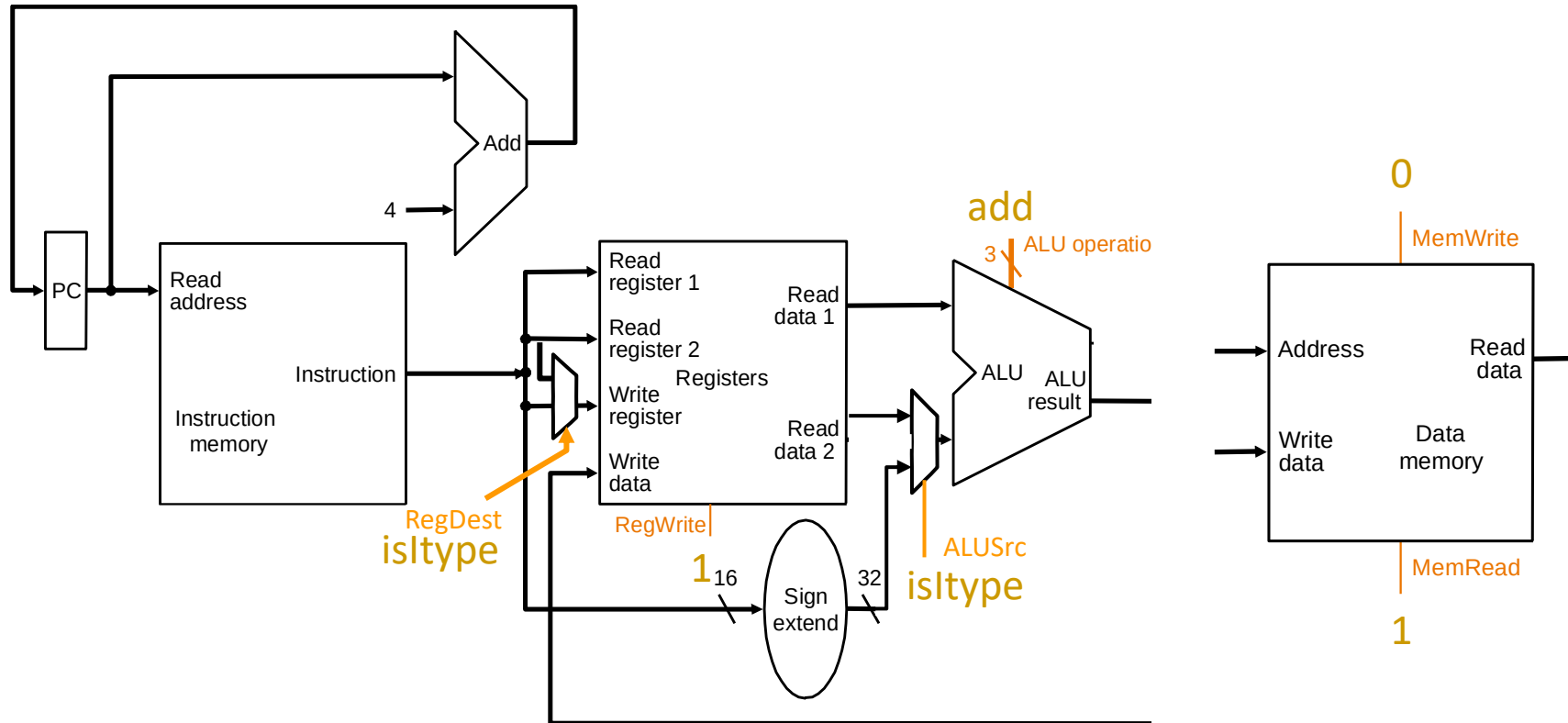
if $\text{MEM}[\text{PC}] == \text{lw rt offset}_{16}(\text{base})$

$\text{PC} \leftarrow \text{PC} + 4$

$\text{EA} = \text{sign-extend}(\text{offset}) + \text{GPR}(\text{base})$

$\text{GPR}[\text{rt}] \leftarrow \text{MEM}[\text{translate}(\text{EA})]$

LW Datapath



if $\text{MEM}[\text{PC}] == \text{LW rt offset}_{16} (\text{base})$
 $\text{EA} = \text{sign-extend}(\text{offset}) + \text{GPR}[\text{base}]$
 $\text{GPR}[\text{rt}] \leftarrow \text{MEM}[\text{translate}(\text{EA})]$
 $\text{PC} \leftarrow \text{PC} + 4$



IF	ID	EX	MEM	WB
----	----	----	-----	----

Combinational
state update logic

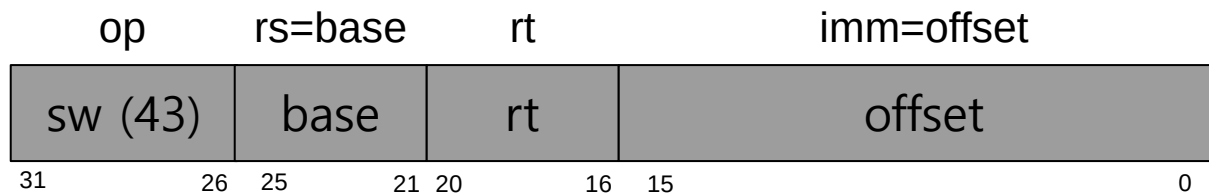
Store Instructions

- Store 4-byte word

MIPS assembly

```
sw    $s3, 8($s0) # $s0=rs, $s3=rt
```

Machine Encoding



I-Type

- Semantics

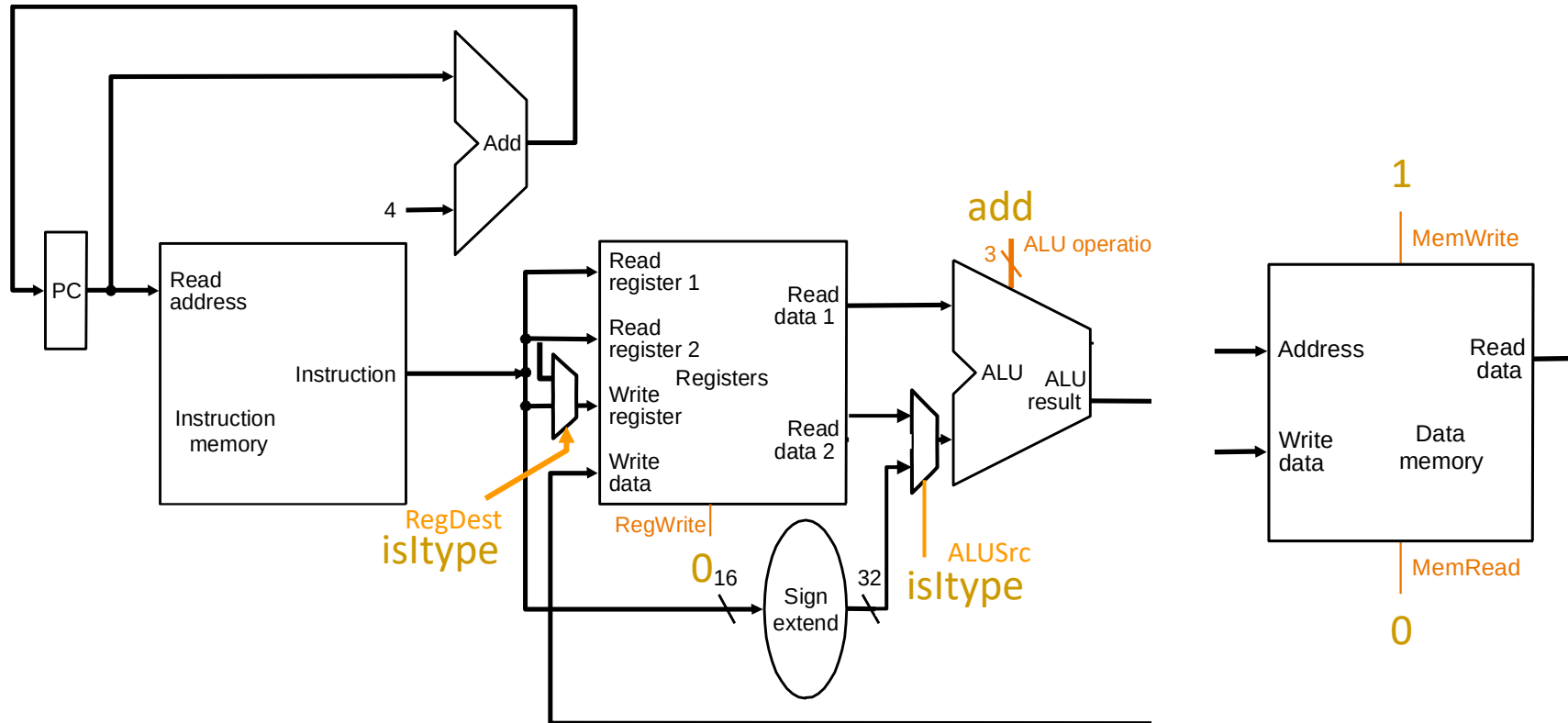
if $\text{Mem}[\text{PC}] == \text{sw rt offset}_{16}(\text{base})$

$\text{PC} \leftarrow \text{PC} + 4$

$\text{EA} = \text{sign-extend}(\text{offset}) + \text{GPR}(\text{base})$

$\text{MEM}[\text{translate}(\text{EA})] \leftarrow \text{GPR}[\text{rt}]$

SW Datapath



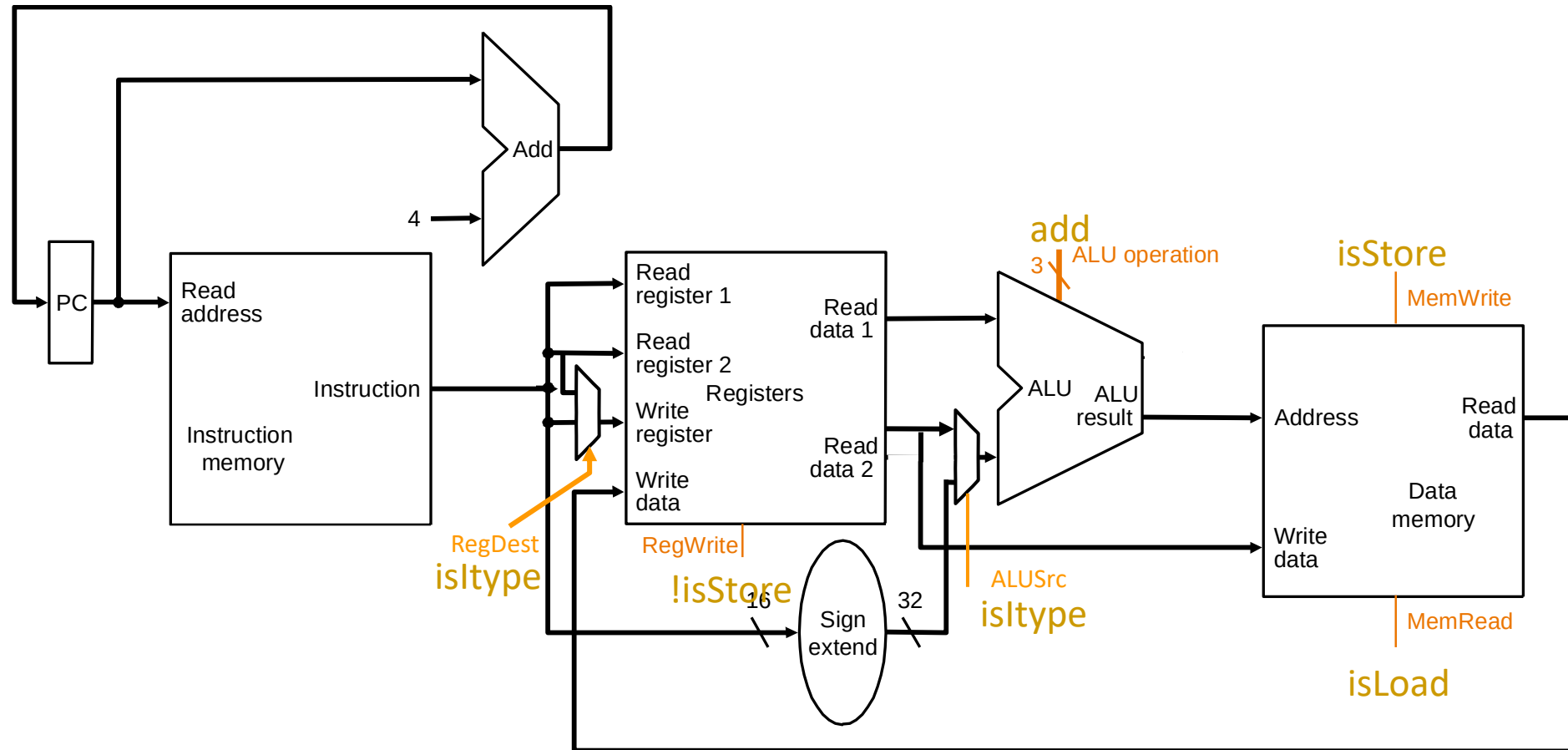
if $\text{MEM}[\text{PC}] == \text{SW rt offset}_{16}(\text{base})$
 $\text{EA} = \text{sign-extend}(\text{offset}) + \text{GPR}[\text{base}]$
 $\text{MEM}[\text{translate}(\text{EA})] \leftarrow \text{GPR}[\text{rt}]$
 $\text{PC} \leftarrow \text{PC} + 4$



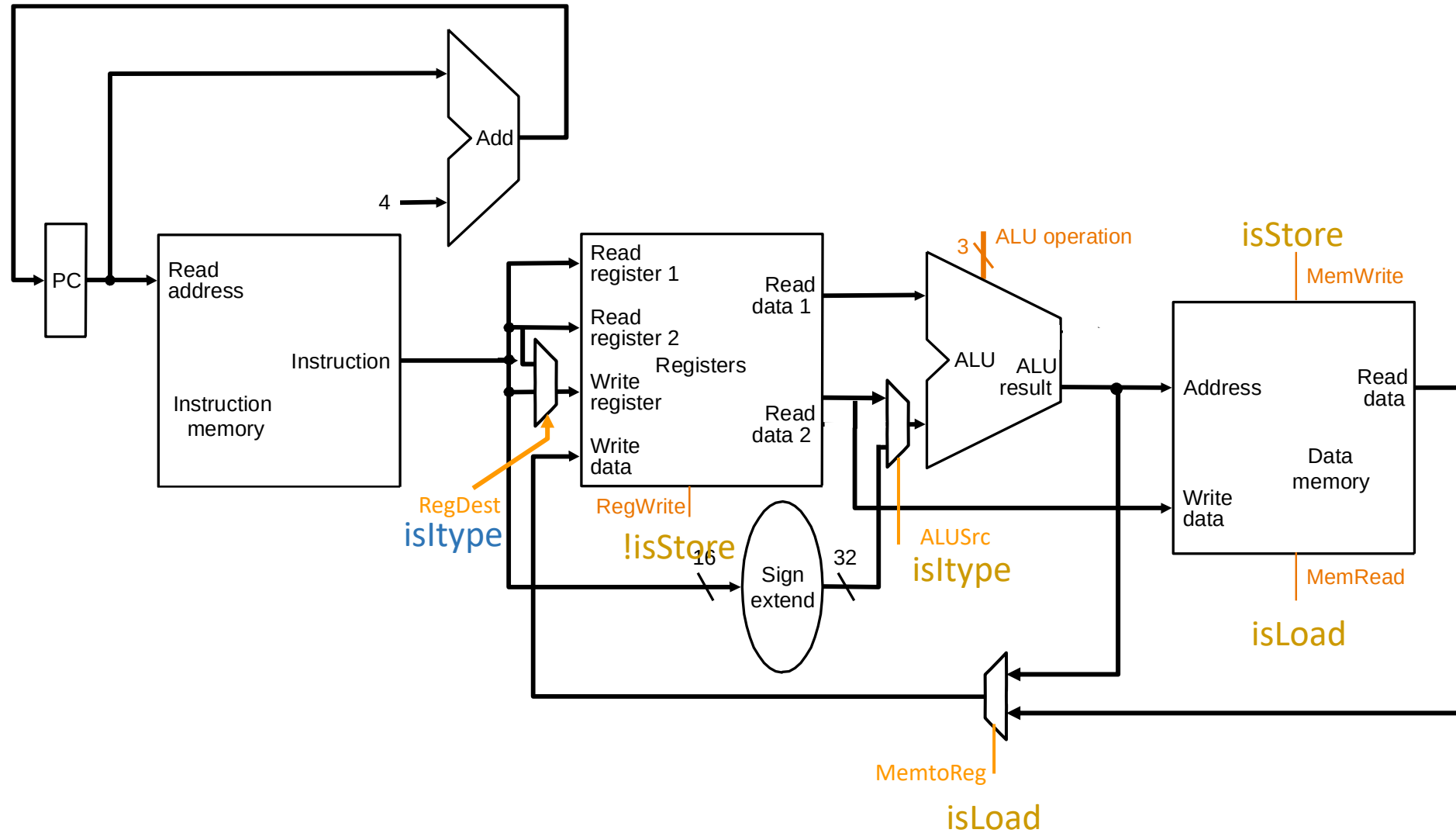
IF	ID	EX	MEM	WB
----	----	----	-----	----

Combinational
state update logic

Load-Store Datapath



Datapath for Non-Control-Flow Insts.

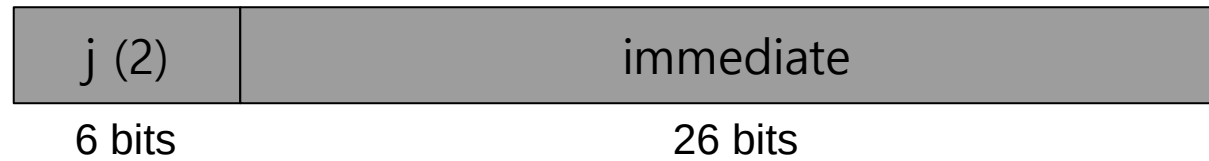


Single-Cycle Datapath for *Control Flow Instructions*

Jump Instruction

- Unconditional branch or jump

j target

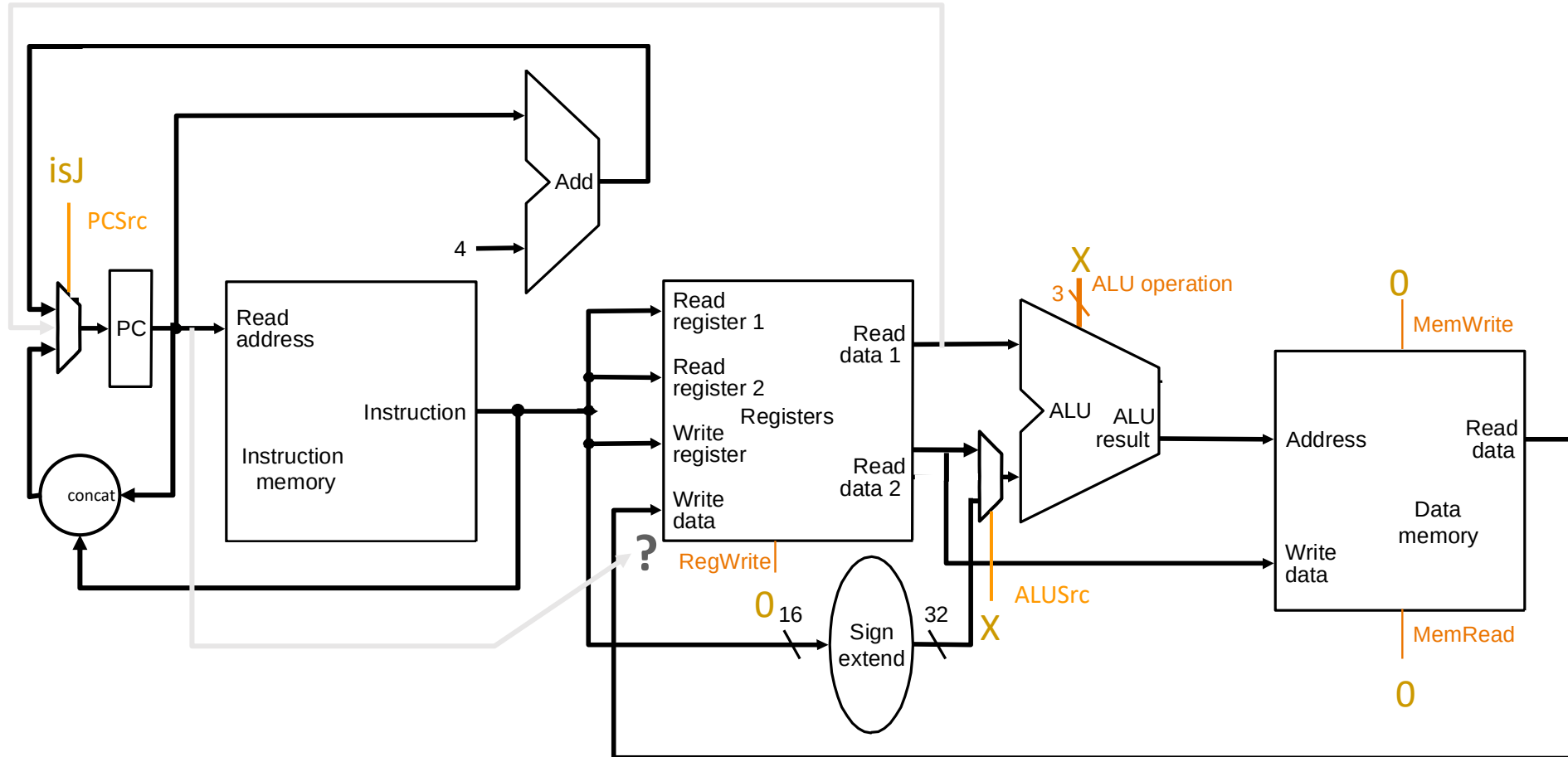


J-Type

- 2 = opcode
 - immediate (target) = target address
-
- Semantics
 - if $\text{MEM}[\text{PC}] == j \text{ immediate}_{26}$
 - target = { PC $\dagger$ [31:28], immediate₂₆, 2' b00 }
 - PC $\leftarrow$ target

$\dagger$ This is the incremented PC

Unconditional Jump Datapath



if MEM[PC]==J immediate26
PC = { PC[31:28], immediate26, 2' b00 }

What about JR, JAL, JALR?

Other Jumps in MIPS

- jal: jump and link (function calls)

- Semantics

- if $\text{MEM}[\text{PC}] == \text{jal immediate}_{26}$

- $\$ra \leftarrow \text{PC} + 4$

- $\text{target} = \{ \text{PC}^\dagger[31:28], \text{immediate}_{26}, 2'b00 \}$

- $\text{PC} \leftarrow \text{target}$

- jr: jump register

- Semantics

- if $\text{MEM}[\text{PC}] == \text{jr rs}$

- $\text{PC} \leftarrow \text{GPR}(\text{rs})$

- jalr: jump and link register

- Semantics

- if $\text{MEM}[\text{PC}] == \text{jalr rs}$

- $\$ra \leftarrow \text{PC} + 4$

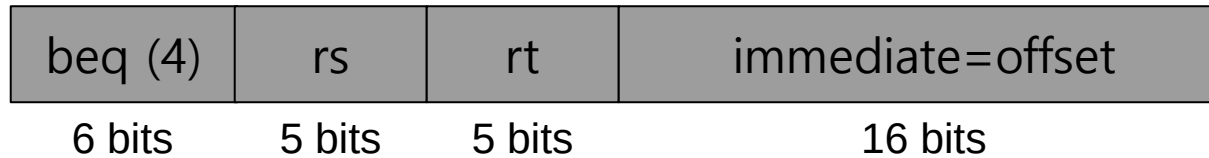
- $\text{PC} \leftarrow \text{GPR}(\text{rs})$

[†] This is the incremented PC

Conditional Branch Instructions

■ beq (Branch if Equal)

```
beq  $s0, $s1, offset  # $s0=rs, $s1=rt
```



I-Type

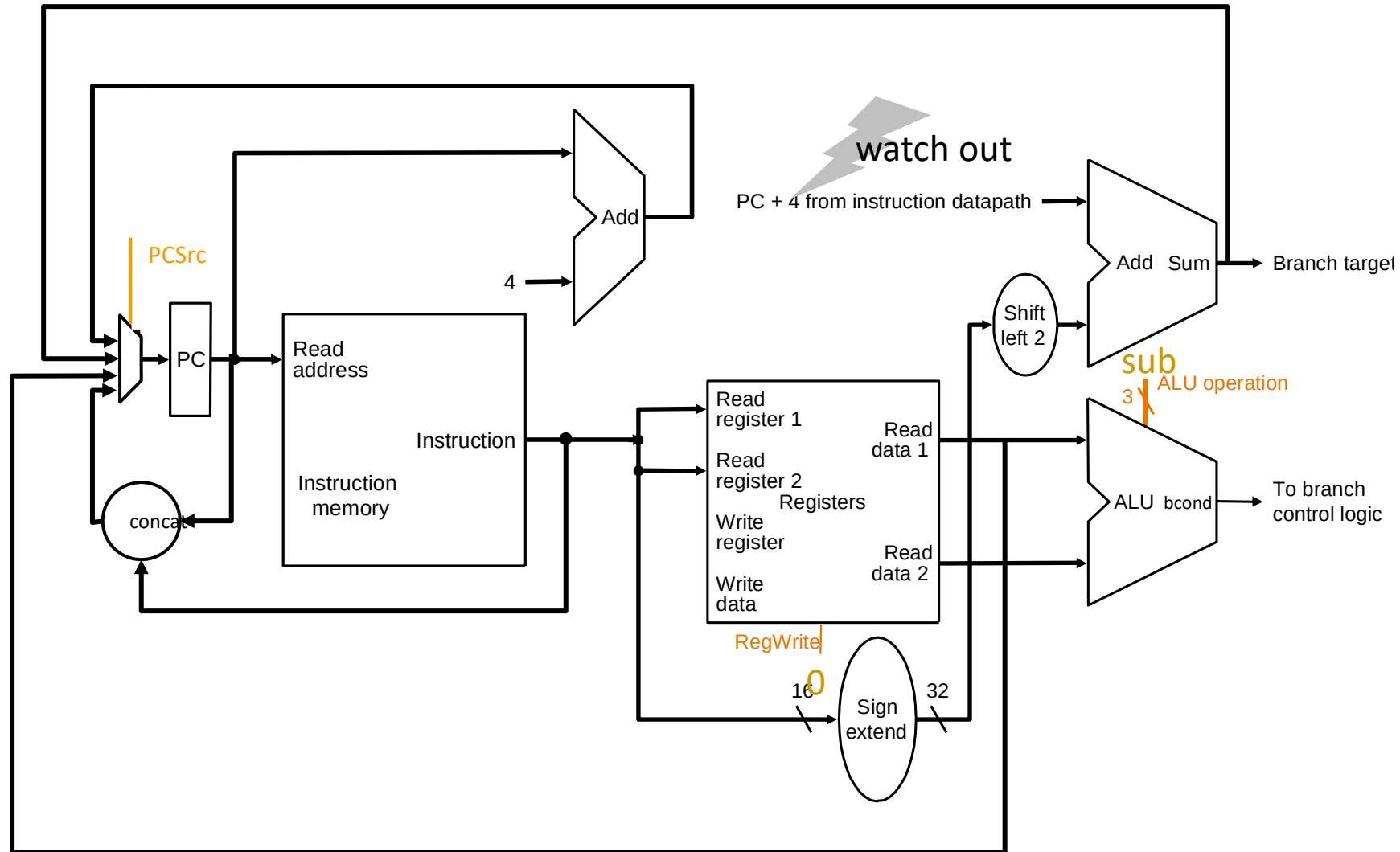
■ Semantics (assuming no branch delay slot)

```
if MEM[PC] == beq rs rt immediate16  
    target = PC† + sign-extend(immediate) x 4  
    if GPR[rs] == GPR[rt] then PC ← target  
    else PC ← PC + 4
```

□ Variations: beq, bne, blez, bgtz

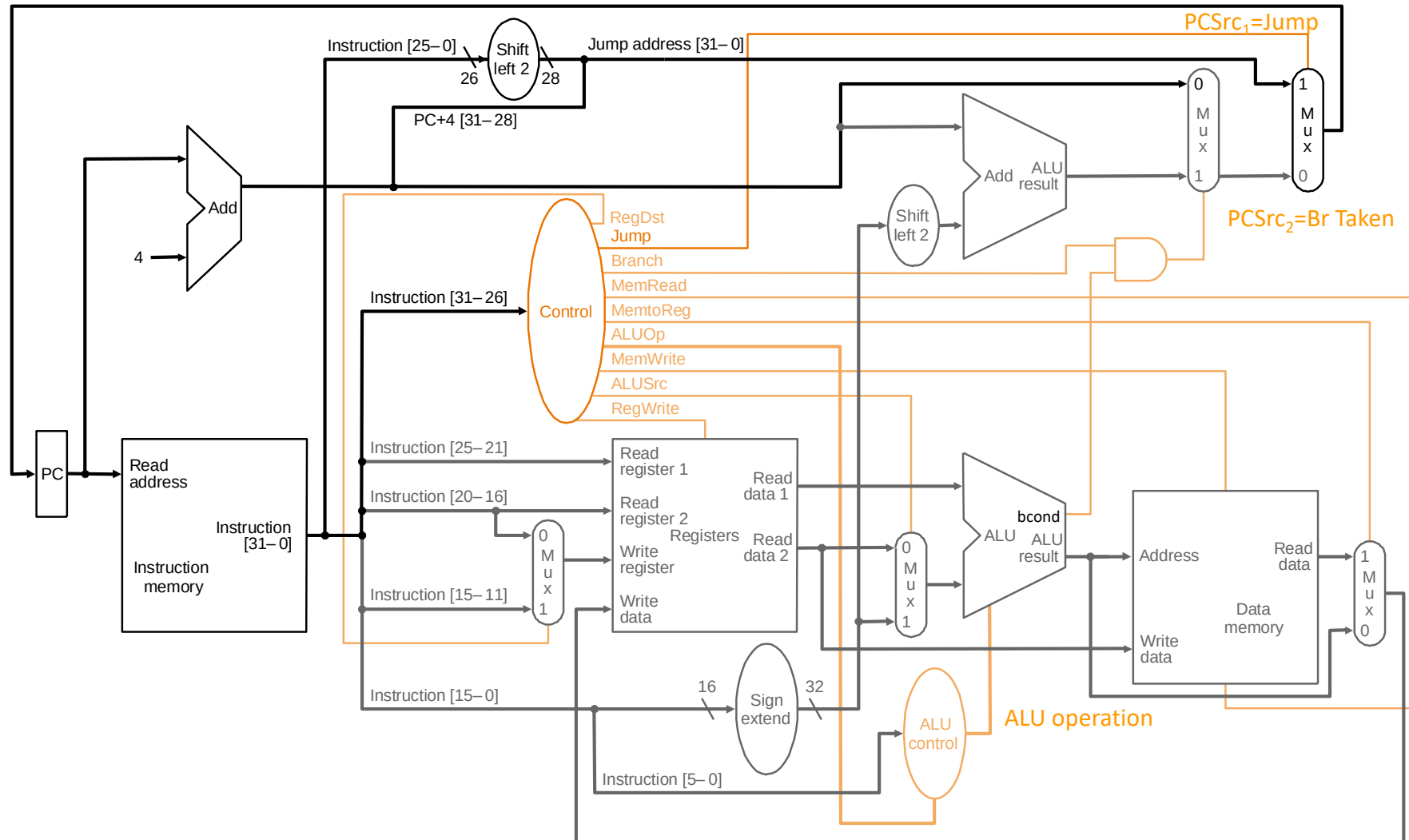
[†] This is the incremented PC

Conditional Branch Datapath (for you to finish)



How to uphold the delayed branch semantics?

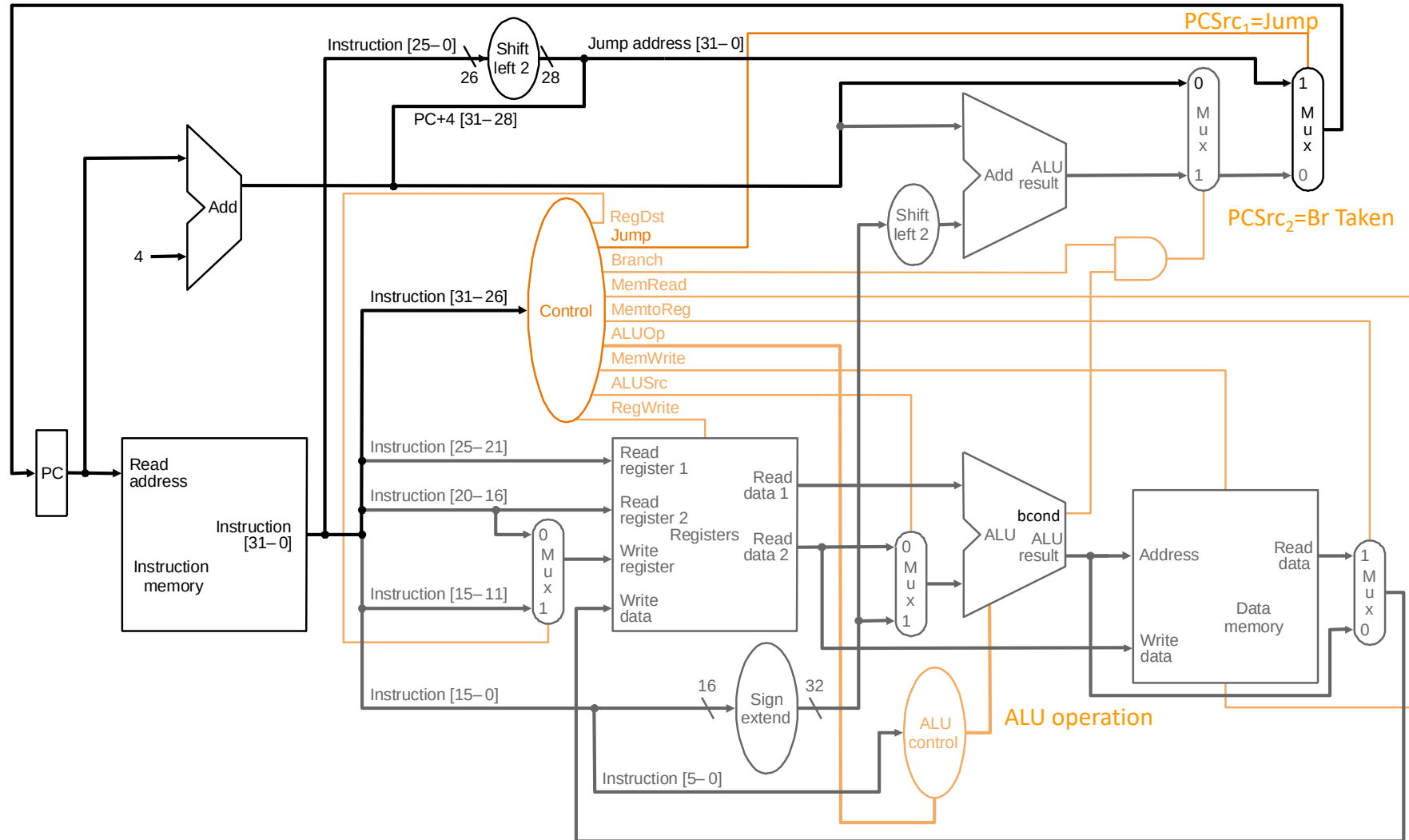
Putting It All Together



JAL, JR, JALR omitted

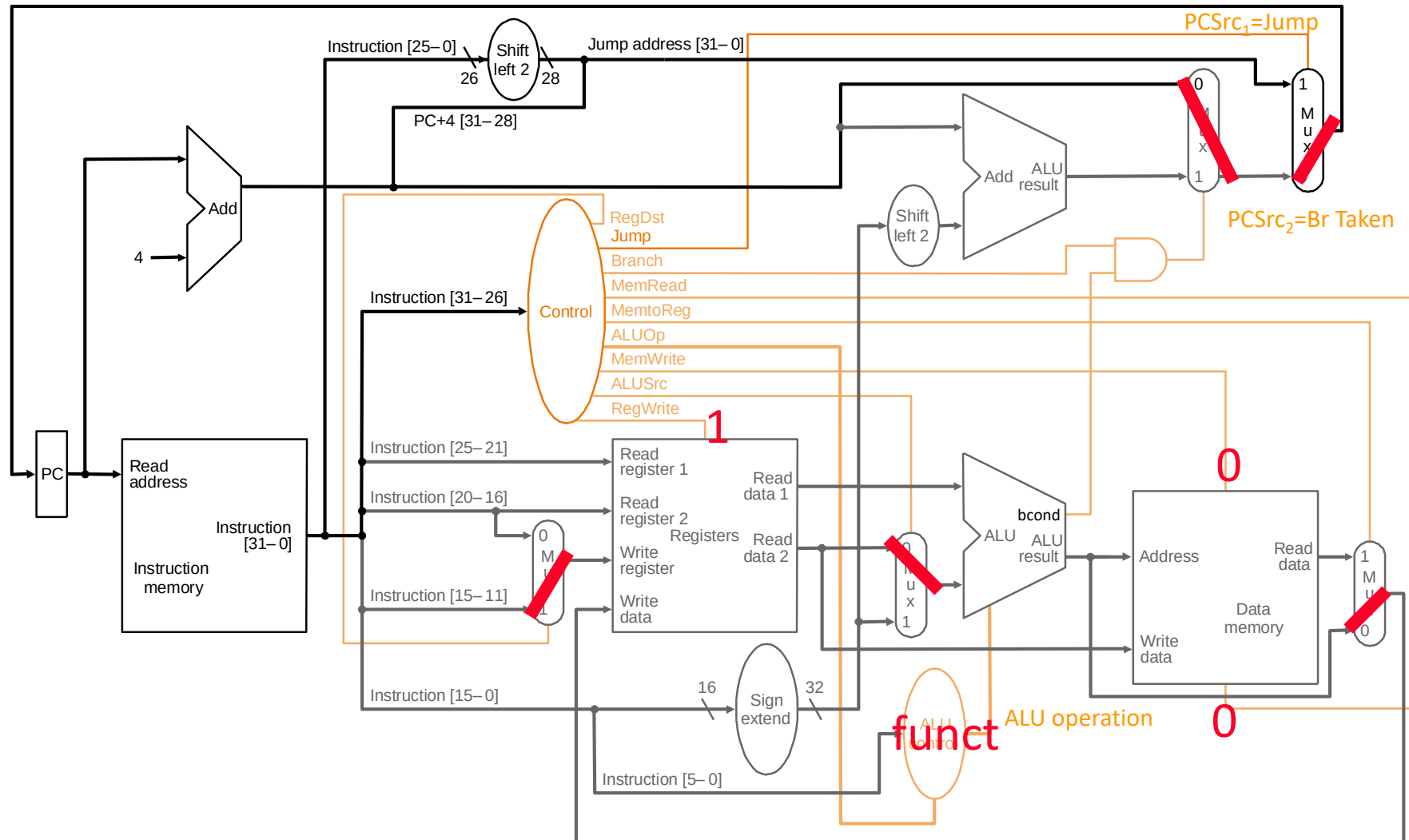
Single-Cycle Control Logic

Let's Control The Single-Cycle MIPS Datapath

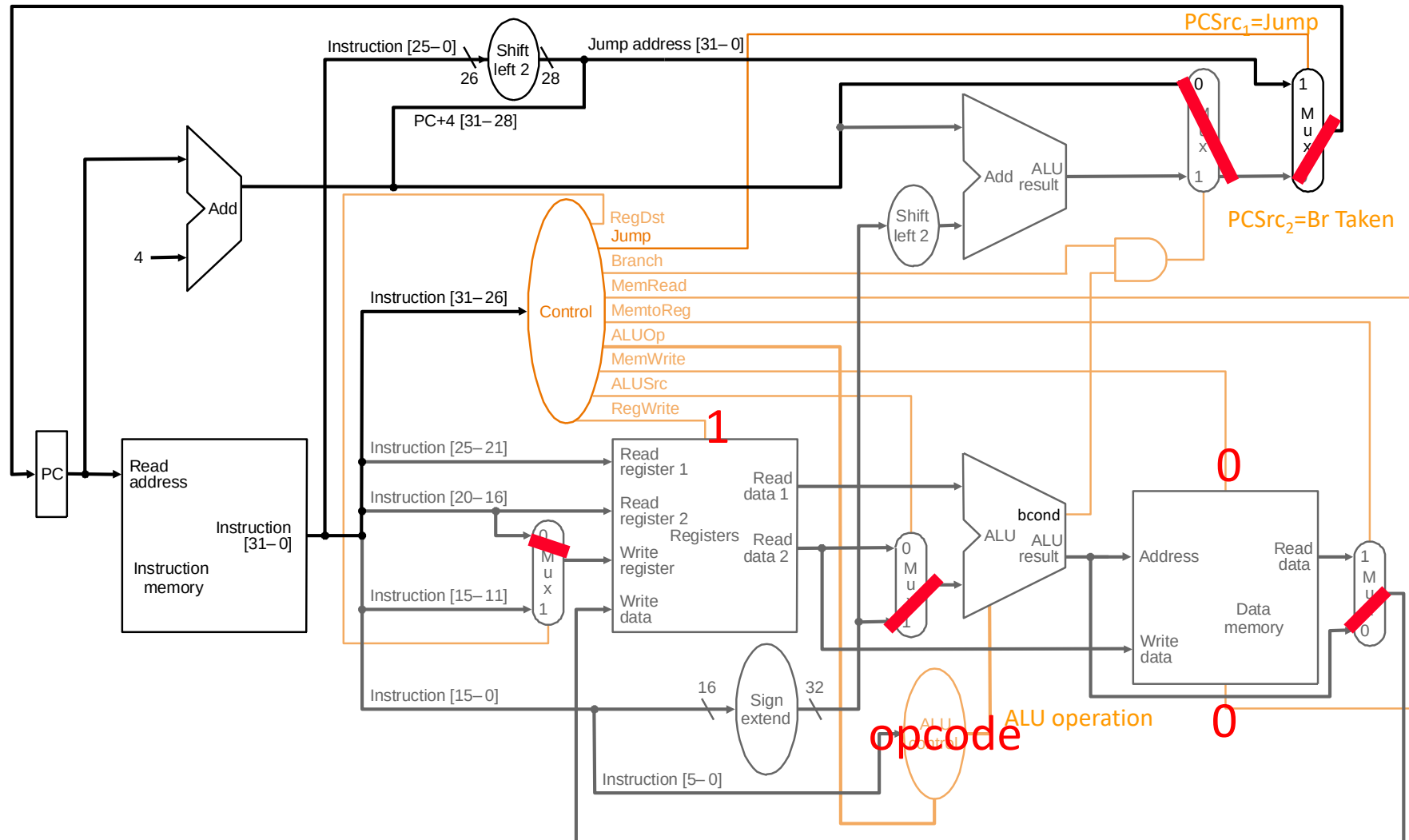


JAL, JR, JALR omitted

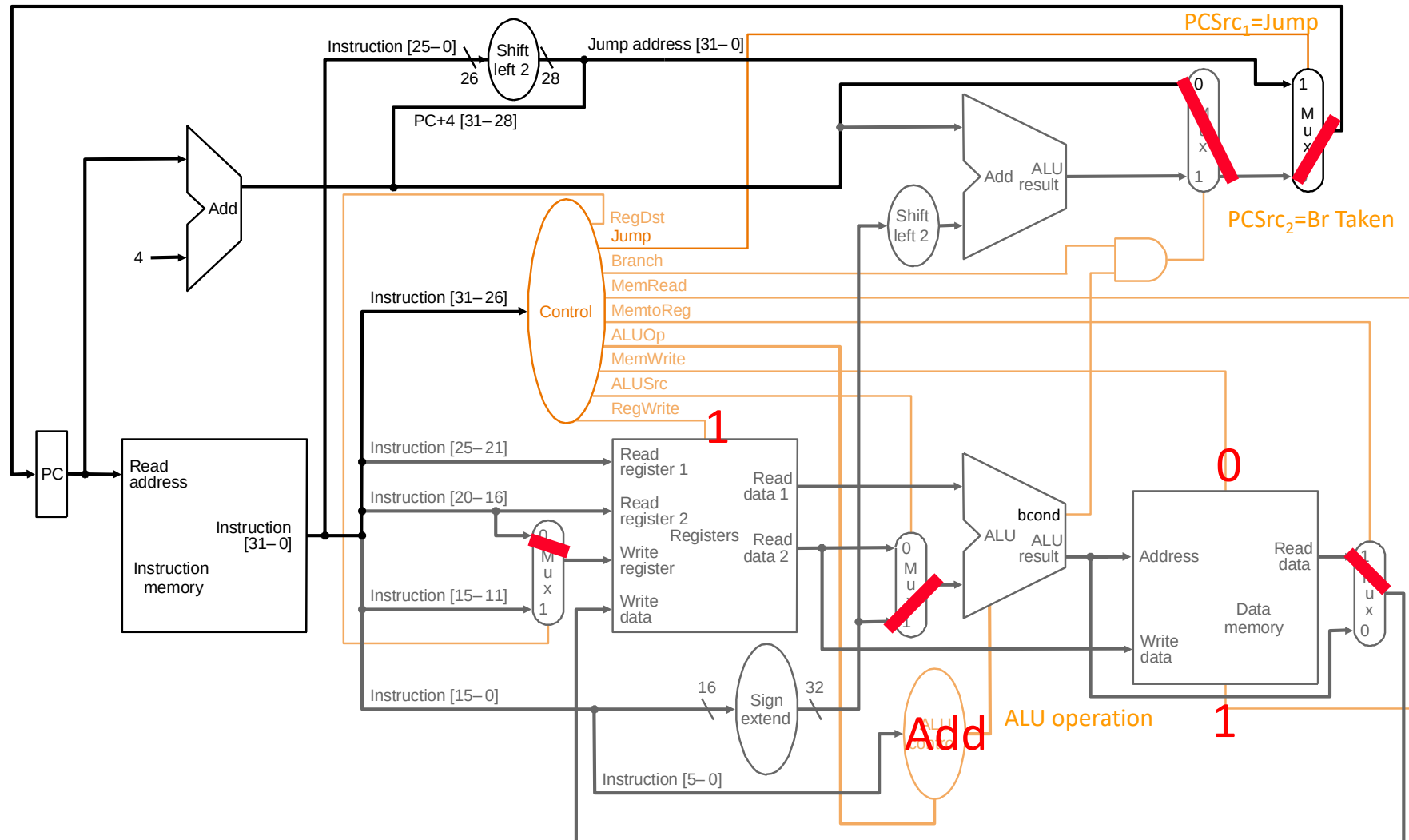
R-Type ALU



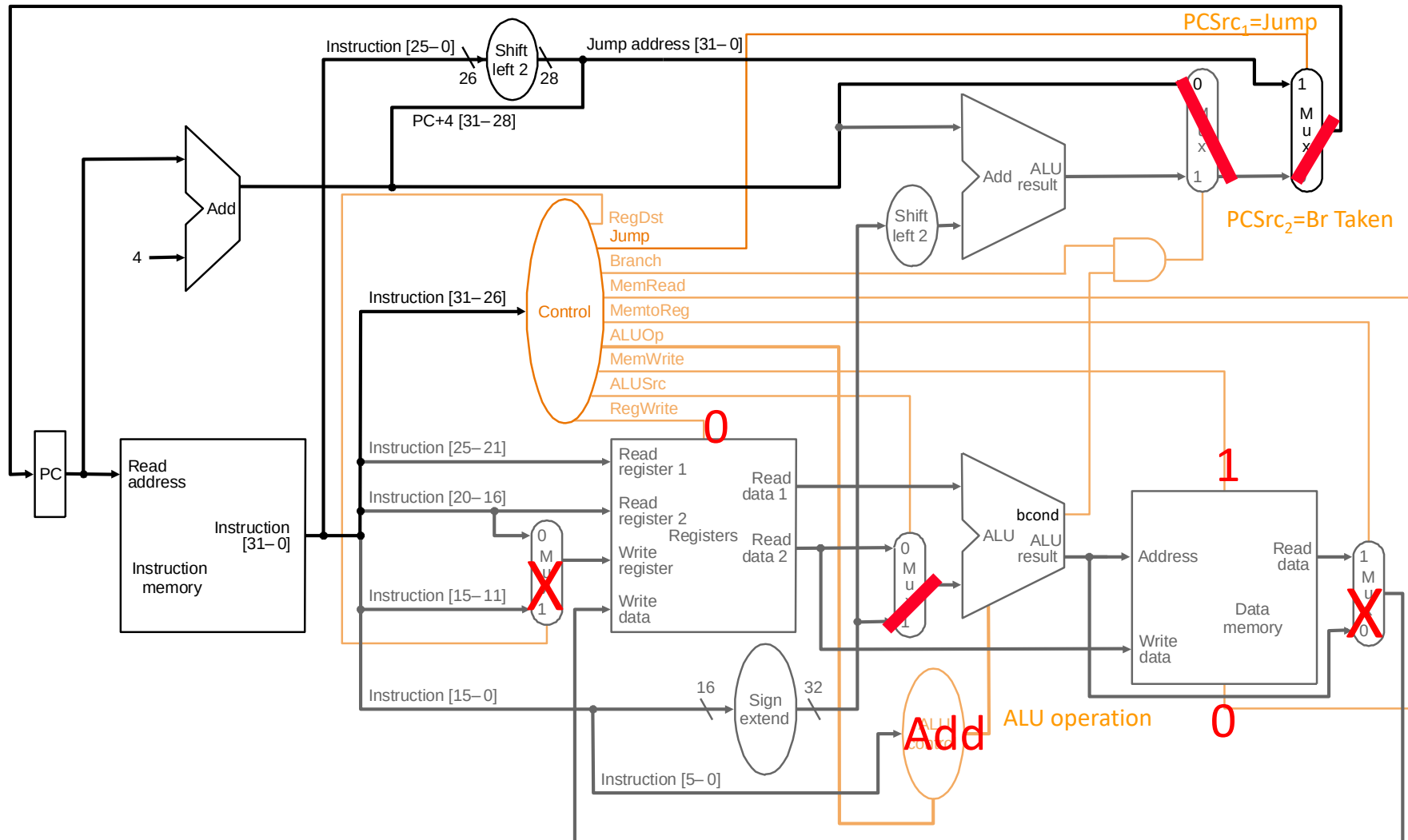
I-Type ALU



LW

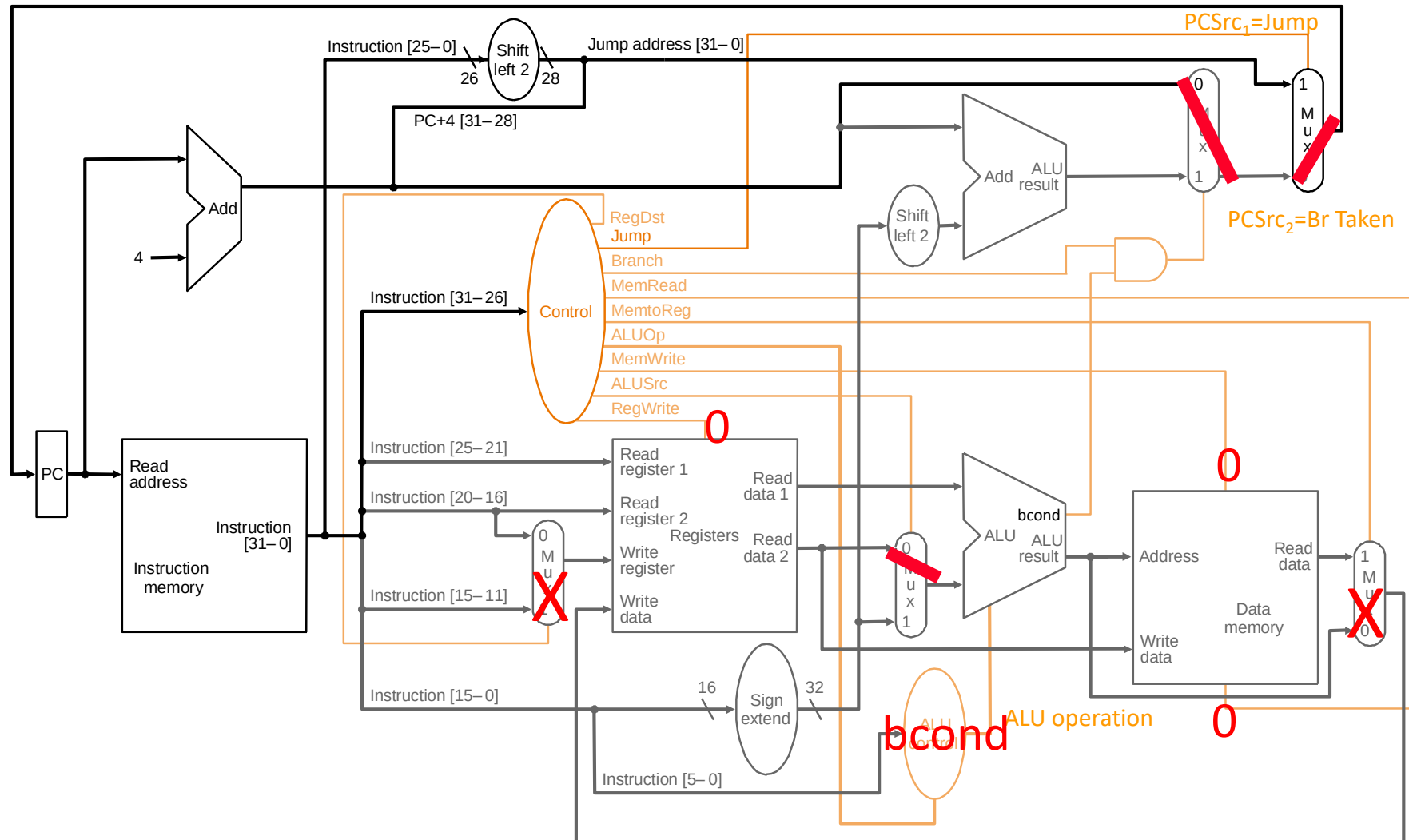


SW



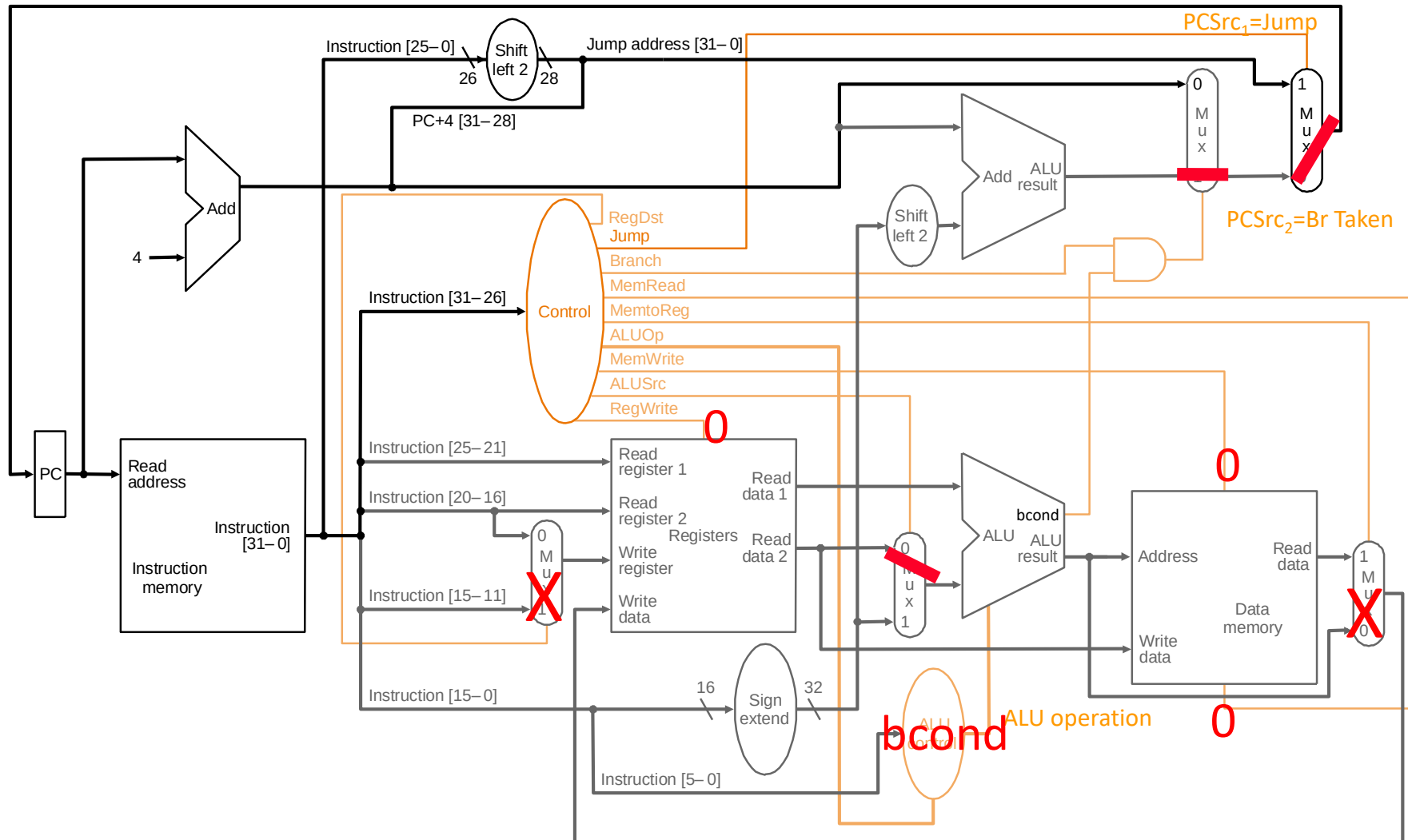
Branch (Not Taken)

Some control signals are dependent on the processing of data

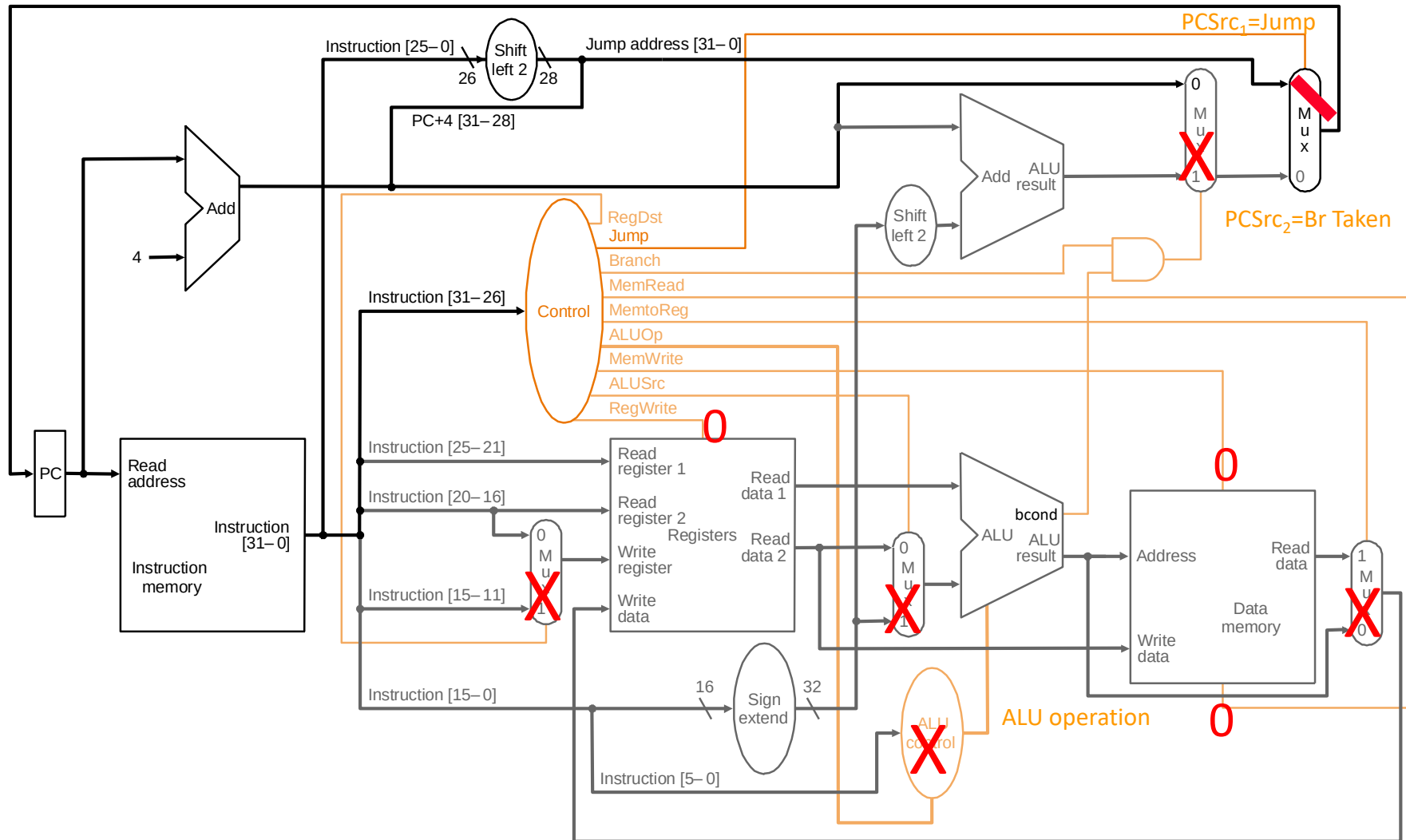


Branch (Taken)

Some control signals are dependent on the processing of data



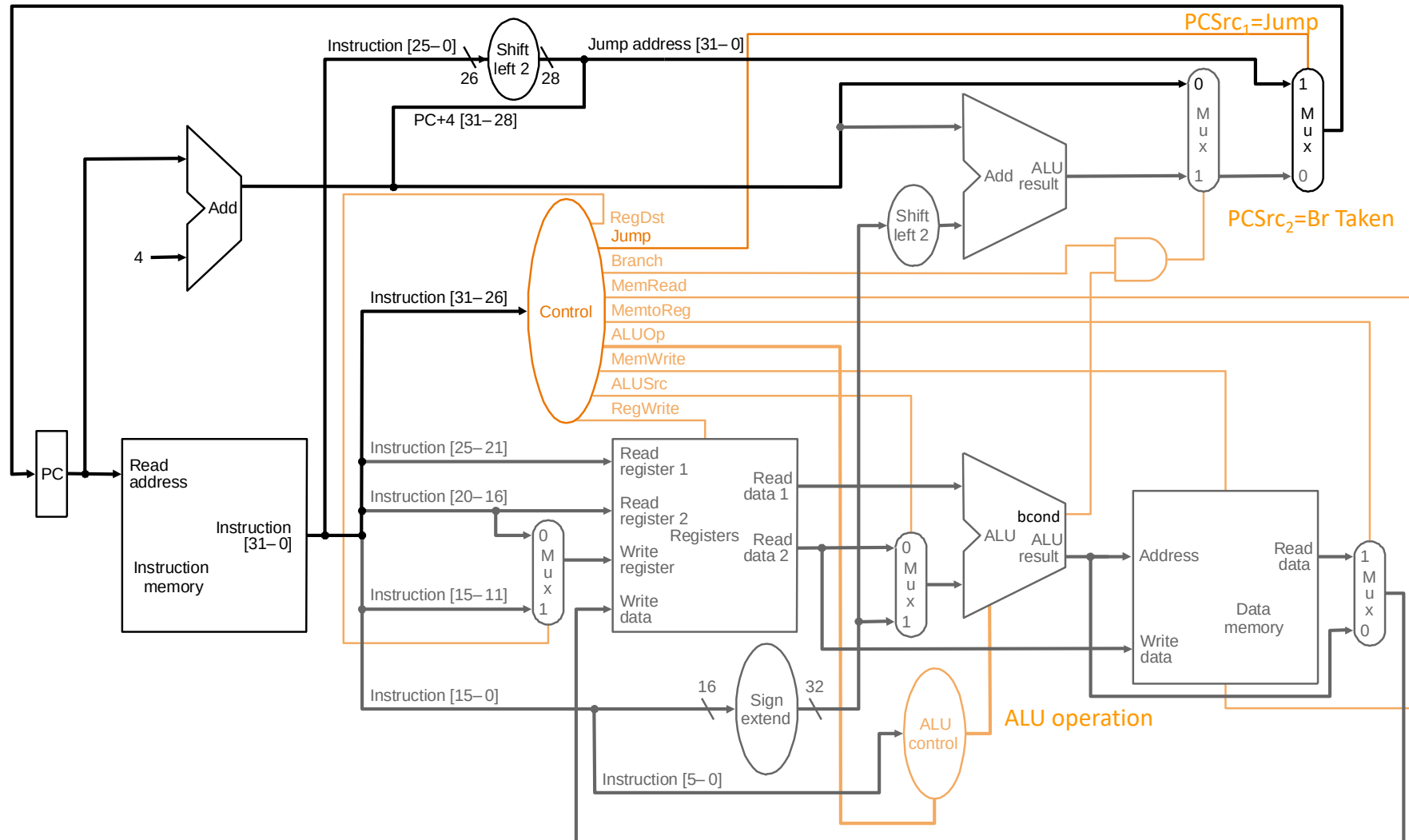
Jump



What is in That Control Box?

- Combinational Logic → **Hardwired Control**
 - Idea: Control signals generated combinatorially based on instruction
 - Necessary in a single-cycle microarchitecture
- Sequential Logic → **Sequential/Microprogrammed Control**
 - Idea: A memory structure contains the control signals associated with an instruction
 - Control Store

Review: Complete Single-Cycle Processor



JAL, JR, JALR omitted

Evaluating the Single-Cycle Microarchitecture

A Single-Cycle Microarchitecture

- Is *this* a good idea/design?
- When is this a good design?
- When is this a bad design?
- How can we design a better microarchitecture?

Performance Analysis Basics

Processor Performance

- How fast is my program?
 - Every program consists of a series of instructions
 - Each instruction needs to be executed.

Processor Performance

- How fast is my program?
 - Every program consists of a series of instructions
 - Each instruction needs to be executed.
- So how fast are my instructions ?
 - Instructions are realized on the hardware
 - They can take one or more clock cycles to complete
 - *Cycles per Instruction = CPI*

Processor Performance

- How fast is my program?
 - Every program consists of a series of instructions
 - Each instruction needs to be executed.
- So how fast are my instructions ?
 - Instructions are realized on the hardware
 - They can take one or more clock cycles to complete
 - *Cycles per Instruction = CPI*
- How much time is one clock cycle?
 - The critical path determines how much time one cycle requires = *clock period*.
 - $1/\text{clock period} = \text{clock frequency}$ = how many cycles can be done each second.

Processor Performance

- Now as a general formula
 - Our program consists of executing N instructions.
 - Our processor needs CPI cycles for each instruction.
 - The maximum clock speed of the processor is f , and the clock period is therefore $T=1/f$

Processor Performance

- Now as a general formula
 - Our program consists of executing **N** instructions.
 - Our processor needs **CPI** cycles for each instruction.
 - The maximum clock speed of the processor is **f**, and the clock period is therefore **T**=1/f
- Our program executes in

$$\begin{aligned} & N \times \text{CPI} \times (1/f) = \\ & N \times \text{CPI} \times T \text{ seconds} \end{aligned}$$

Performance Analysis Basics

- Execution time of an instruction
 - $\{\text{CPI}\} \times \{\text{clock cycle time}\}$
 - CPI: Number of cycles it takes to execute an instruction
- Execution time of a program
 - Sum over all instructions $[\{\text{CPI}\} \times \{\text{clock cycle time}\}]$
 - $\{\# \text{ of instructions}\} \times \{\text{Average CPI}\} \times \{\text{clock cycle time}\}$

Performance Analysis of Our Single-Cycle Design

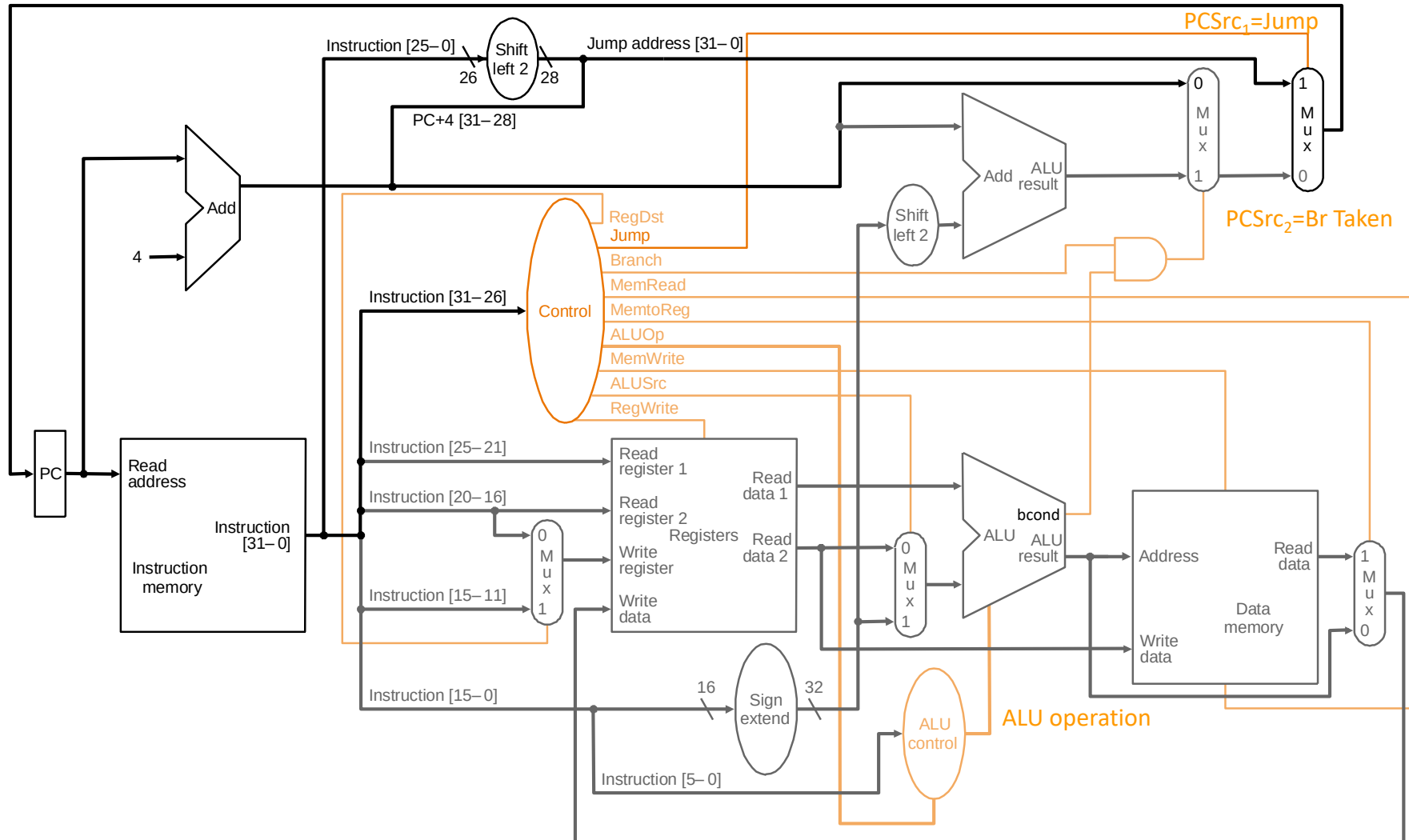
A Single-Cycle Microarchitecture: Analysis

- Every instruction takes 1 cycle to execute
 - CPI (Cycles per instruction) is strictly 1
- How long each instruction takes is determined by how long the slowest instruction takes to execute
 - Even though many instructions do not need that long to execute
- Clock cycle time of the microarchitecture is determined by how long it takes to complete the slowest instruction
 - Critical path of the design is determined by the processing time of the slowest instruction

What is the Slowest Instruction to Process?

- Let's go back to the basics
- All six phases of the instruction processing cycle take a *single machine clock cycle* to complete
 - Fetch
 - 1. Instruction fetch (IF)
 - Decode
 - 2. Instruction decode and register operand fetch (ID/RF)
 - Evaluate Address
 - 3. Execute/Evaluate memory address (EX/AG)
 - Fetch Operands
 - 4. Memory operand fetch (MEM)
 - Execute
 - 5. Store/writeback result (WB)
 - Store Result
- Do each of the above phases take the same time (latency) for all instructions?

Let's Find the Critical Path

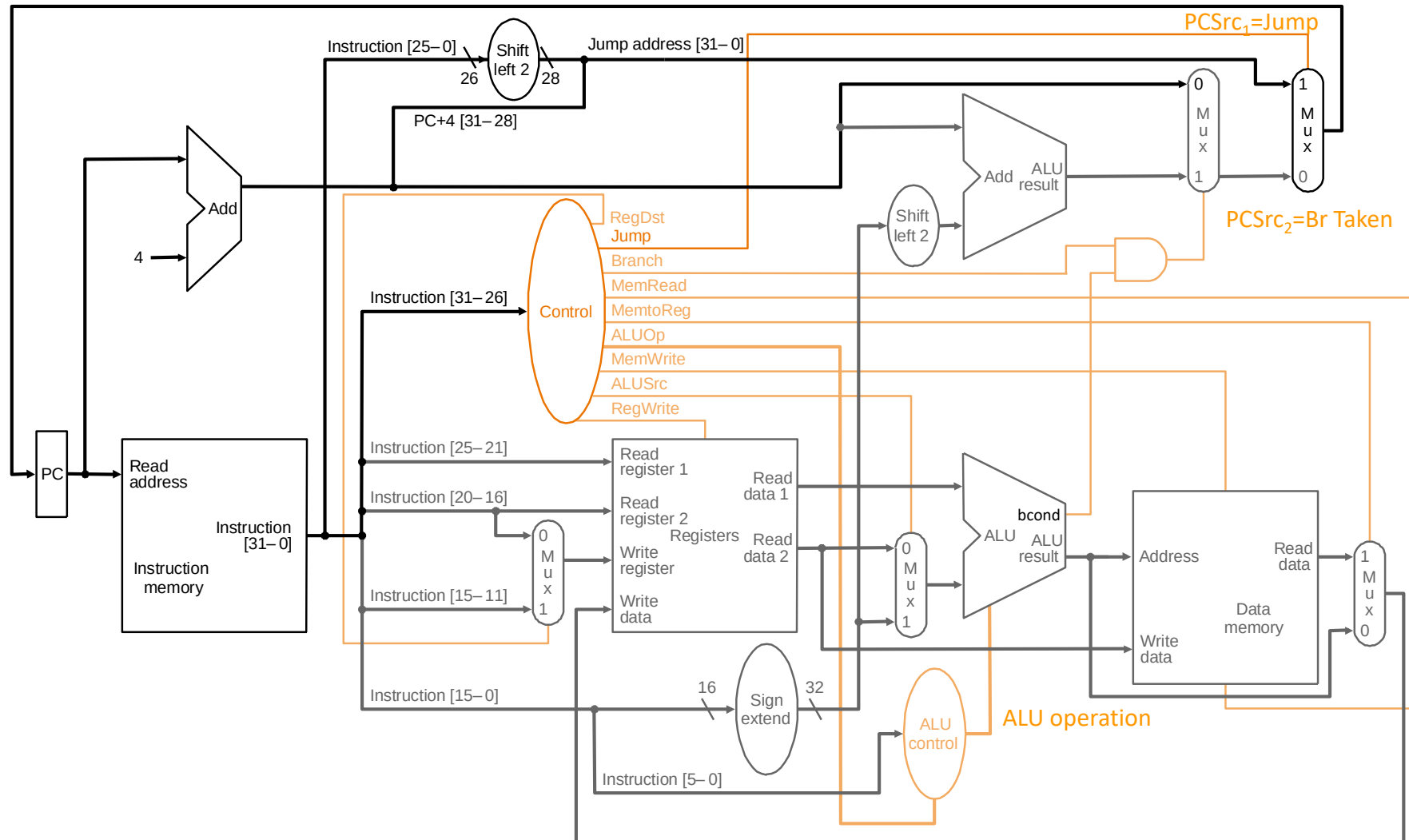


Example Single-Cycle Datapath Analysis

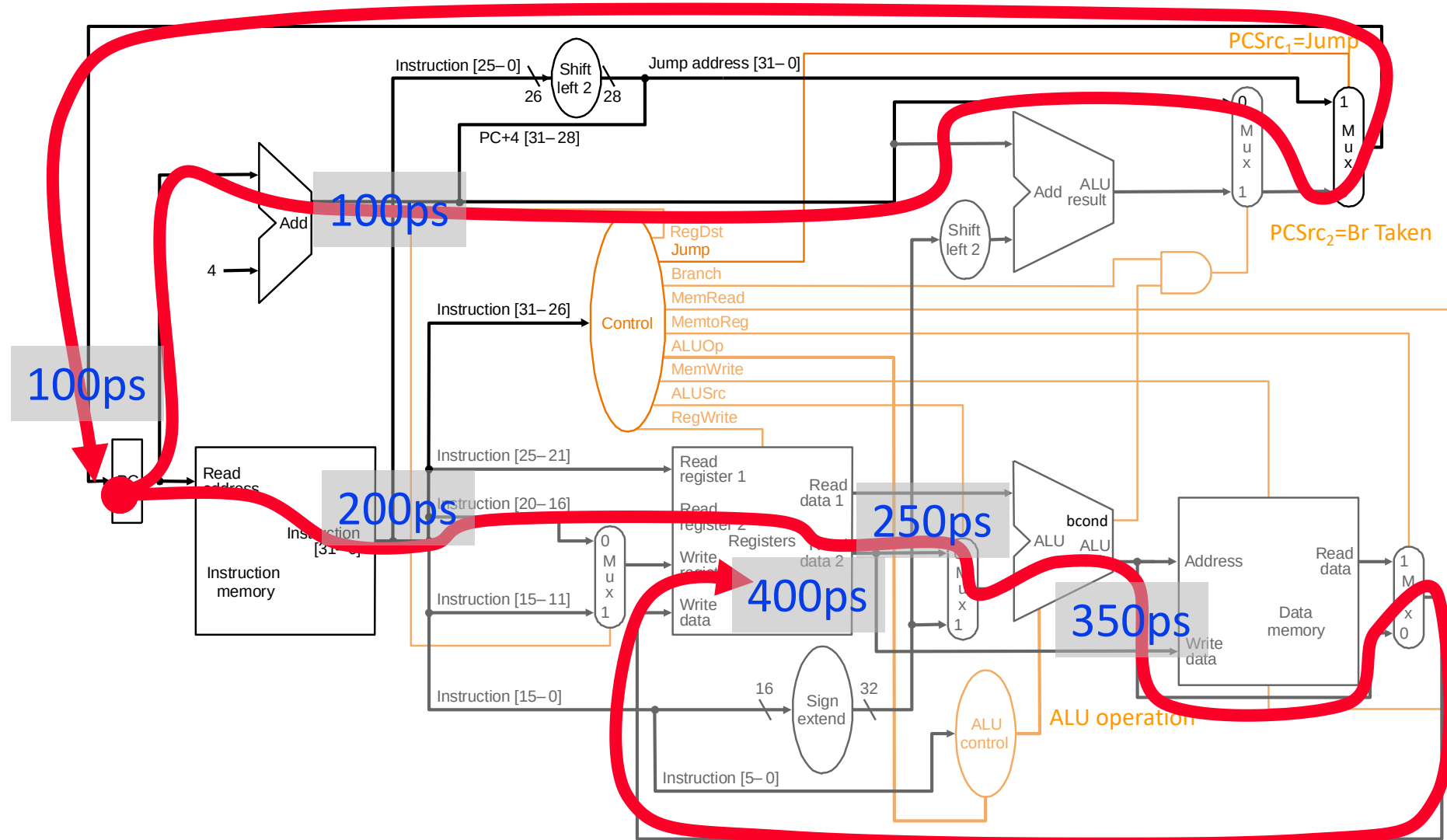
- Assume (for the design in the previous slide)
 - memory units (read or write): 200 ps
 - ALU and adders: 100 ps
 - register file (read or write): 50 ps
 - other combinational logic: 0 ps

steps	IF	ID	EX	MEM	WB	Delay
resources	mem	RF	ALU	mem	RF	
R-type	200	50	100		50	400
I-type	200	50	100		50	400
LW	200	50	100	200	50	600
SW	200	50	100	200		550
Branch	200	50	100			350
Jump	200					200

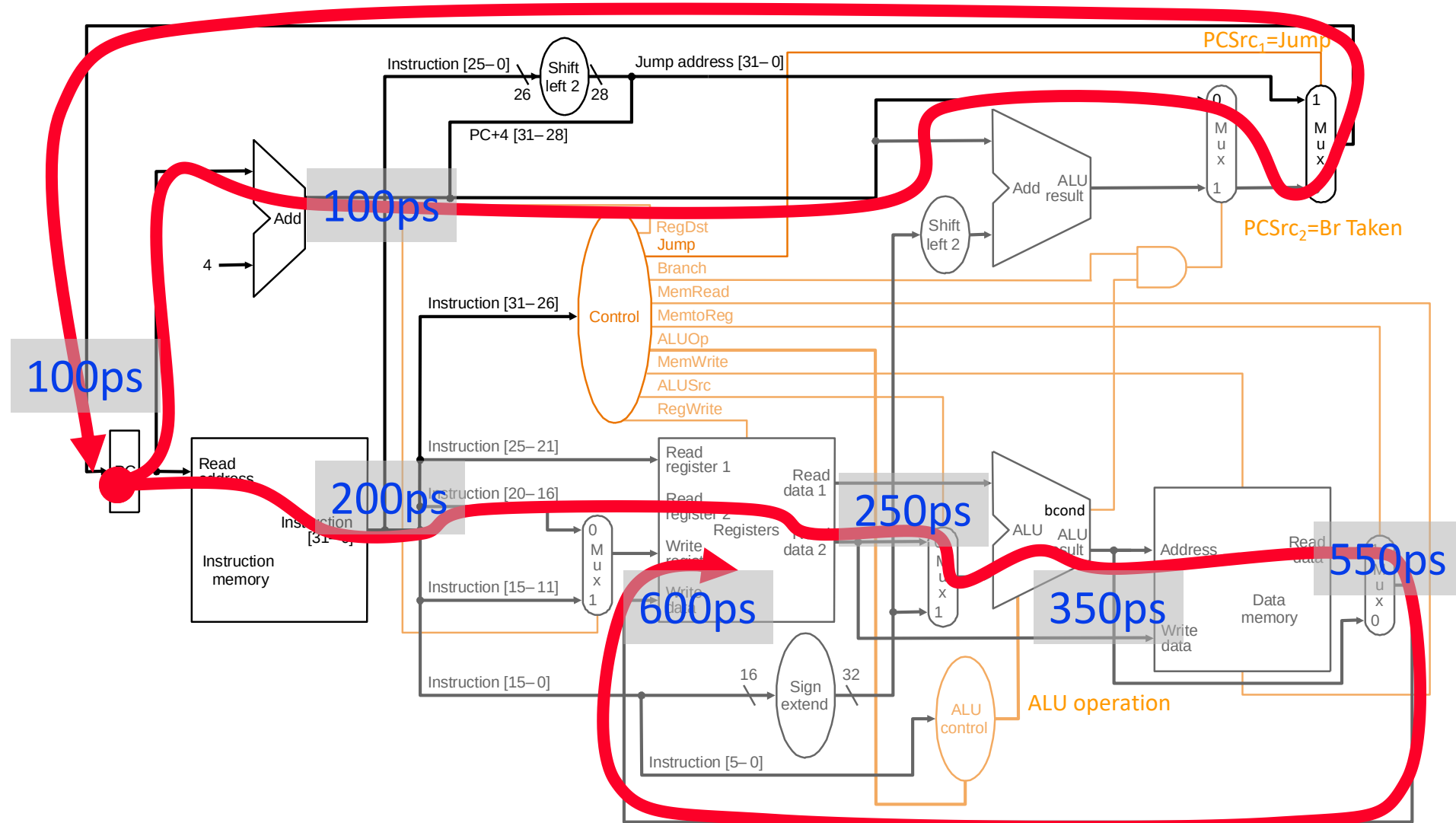
Let's Find the Critical Path



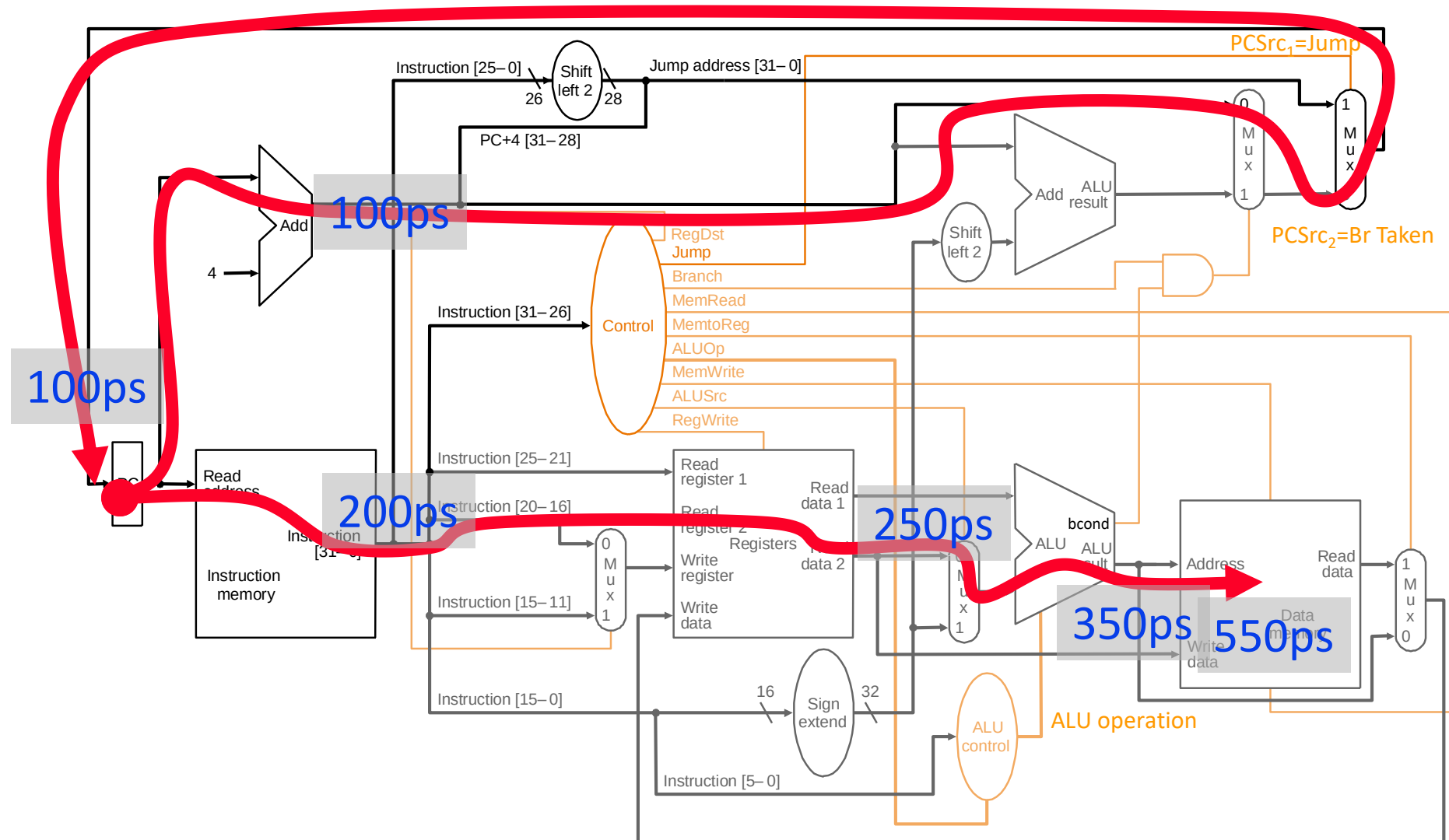
R-Type and I-Type ALU



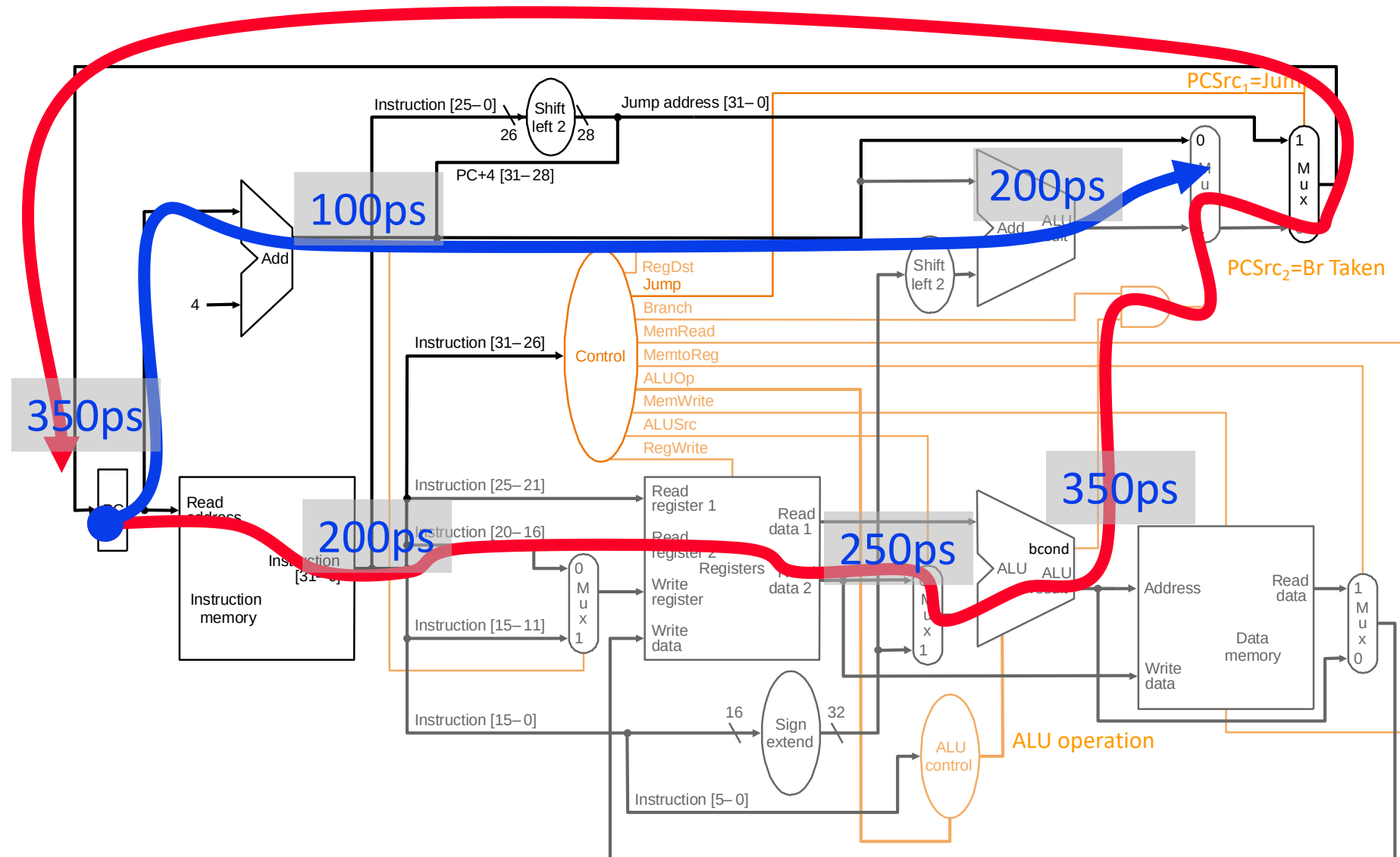
LW



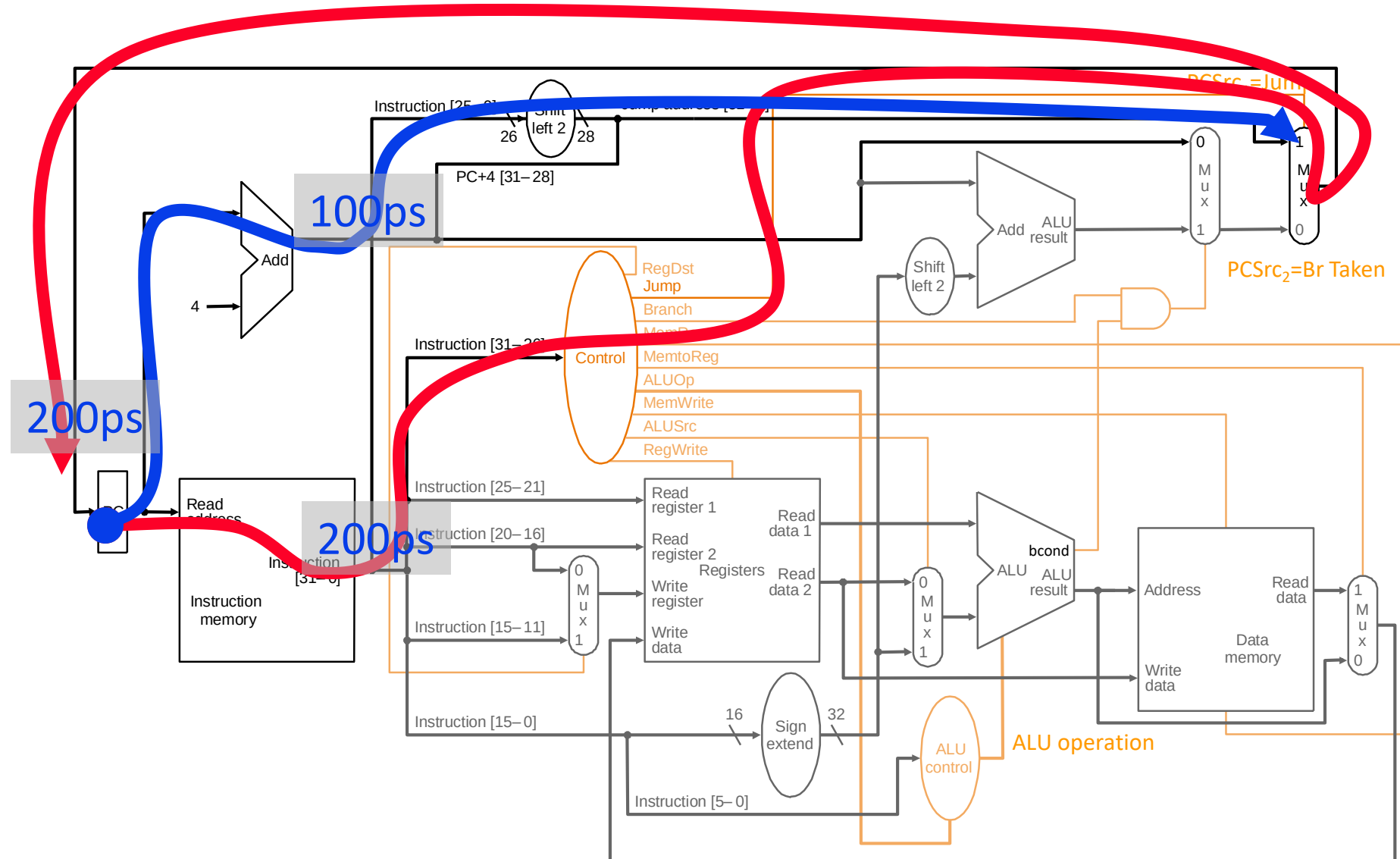
SW



Branch Taken



Jump



What is the Slowest Instruction to Process?

- Memory is not magic
- What if memory *sometimes* takes 100ms to access?
- Does it make sense to have a simple register to register add or jump to take {100ms+all else to do a memory operation}?
- And, what if you need to access memory more than once to process an instruction?
 - Which instructions need this?

Multi-Cycle Microarchitectures

Multi-Cycle Microarchitectures

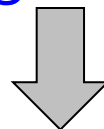
- Goal: Let each instruction take (close to) only as much time it really needs
- Idea
 - Determine clock cycle time independently of instruction processing time
 - Each instruction takes as many clock cycles as it needs to take
 - Multiple state transitions per instruction
 - The states followed by each instruction is different

Remember: The “Process instruction” Step

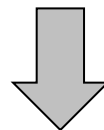
- ISA specifies abstractly what AS' should be, given an instruction and AS
 - It defines an abstract finite state machine where
 - State = programmer-visible state
 - Next-state logic = instruction execution specification
 - From ISA point of view, there are no “intermediate states” between AS and AS' during instruction execution
 - One state transition per instruction
- Microarchitecture implements how AS is transformed to AS'
 - There are many choices in implementation
 - We can have programmer-invisible state to optimize the speed of instruction execution: multiple state transitions per instruction
 - Choice 1: $AS \rightarrow AS'$ (transform AS to AS' in a single clock cycle)
 - Choice 2: $AS \rightarrow AS+MS1 \rightarrow AS+MS2 \rightarrow AS+MS3 \rightarrow AS'$ (take multiple clock cycles to transform AS to AS')

Multi-Cycle Microarchitecture

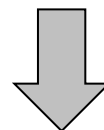
AS = Architectural (programmer visible) state
at the beginning of an instruction



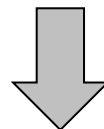
Step 1: Process part of instruction in one clock cycle



Step 2: Process part of instruction in the next clock cycle



...



AS' = Architectural (programmer visible) state
at the end of a clock cycle

Downsides of Multi-Cycle Design

- Need to store the intermediate results at the end of each clock cycle
 - Hardware overhead for registers
 - Register setup/hold overhead paid multiple times for an instruction

Remember: Performance Analysis

- Execution time of an instruction
 - $\{\text{CPI}\} \times \{\text{clock cycle time}\}$
- Execution time of a program
 - Sum over all instructions $[\{\text{CPI}\} \times \{\text{clock cycle time}\}]$
 - $\{\# \text{ of instructions}\} \times \{\text{Average CPI}\} \times \{\text{clock cycle time}\}$
- Single cycle microarchitecture performance
 - $\text{CPI} = 1$
 - Clock cycle time = long

Not easy to optimize design
- Multi-cycle microarchitecture performance
 - CPI = different for each instruction
 - Average CPI $\rightarrow$ hopefully small
 - Clock cycle time = short

We have two degrees of freedom to optimize independently

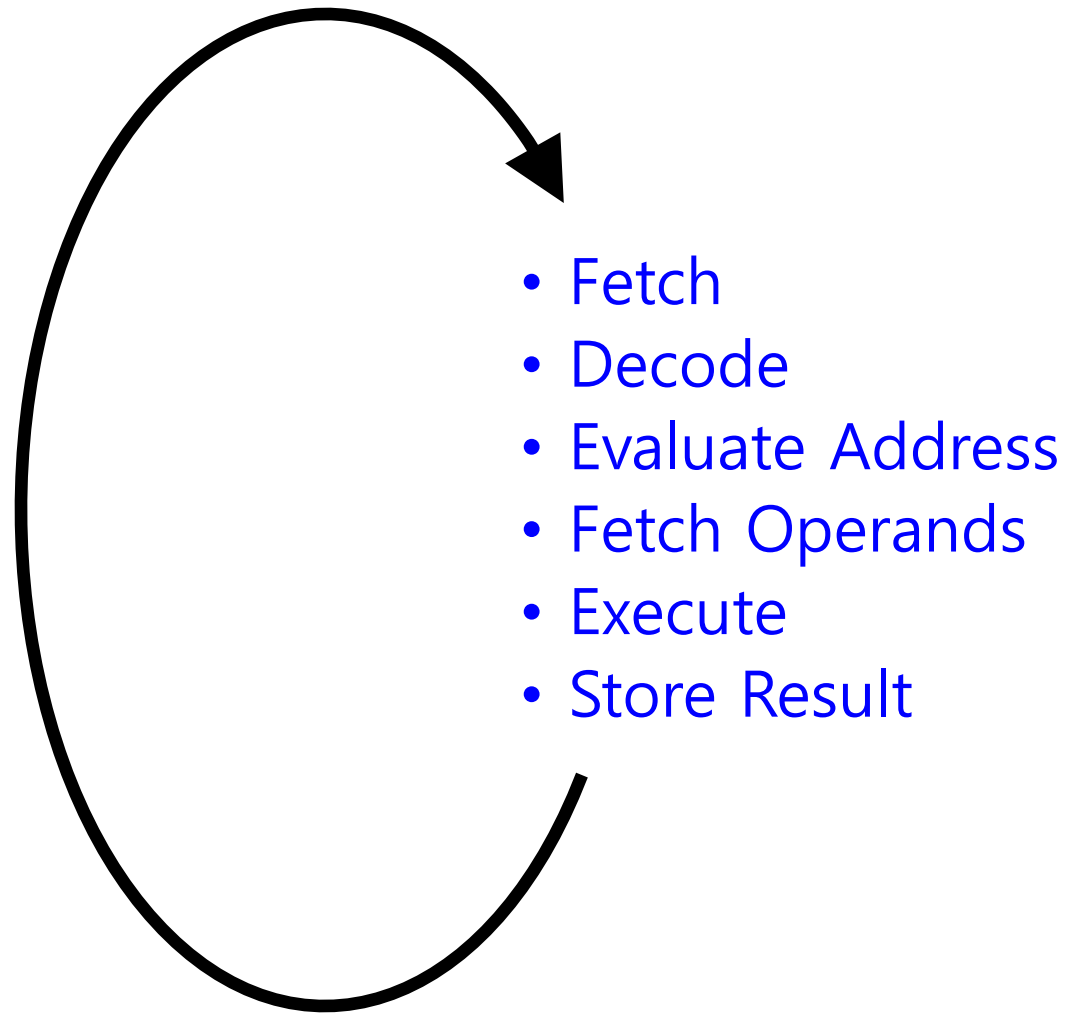
A Multi-Cycle Microarchitecture

A Closer Look

Multi-Cycle uArch

- Key Idea for Realization
 - One can implement the “process instruction” step as a **finite state machine** that sequences between states and eventually returns back to the “fetch instruction” state
 - A state is defined by the control signals asserted in it
 - Control signals for the next state are determined in current state

The Instruction Processing Cycle

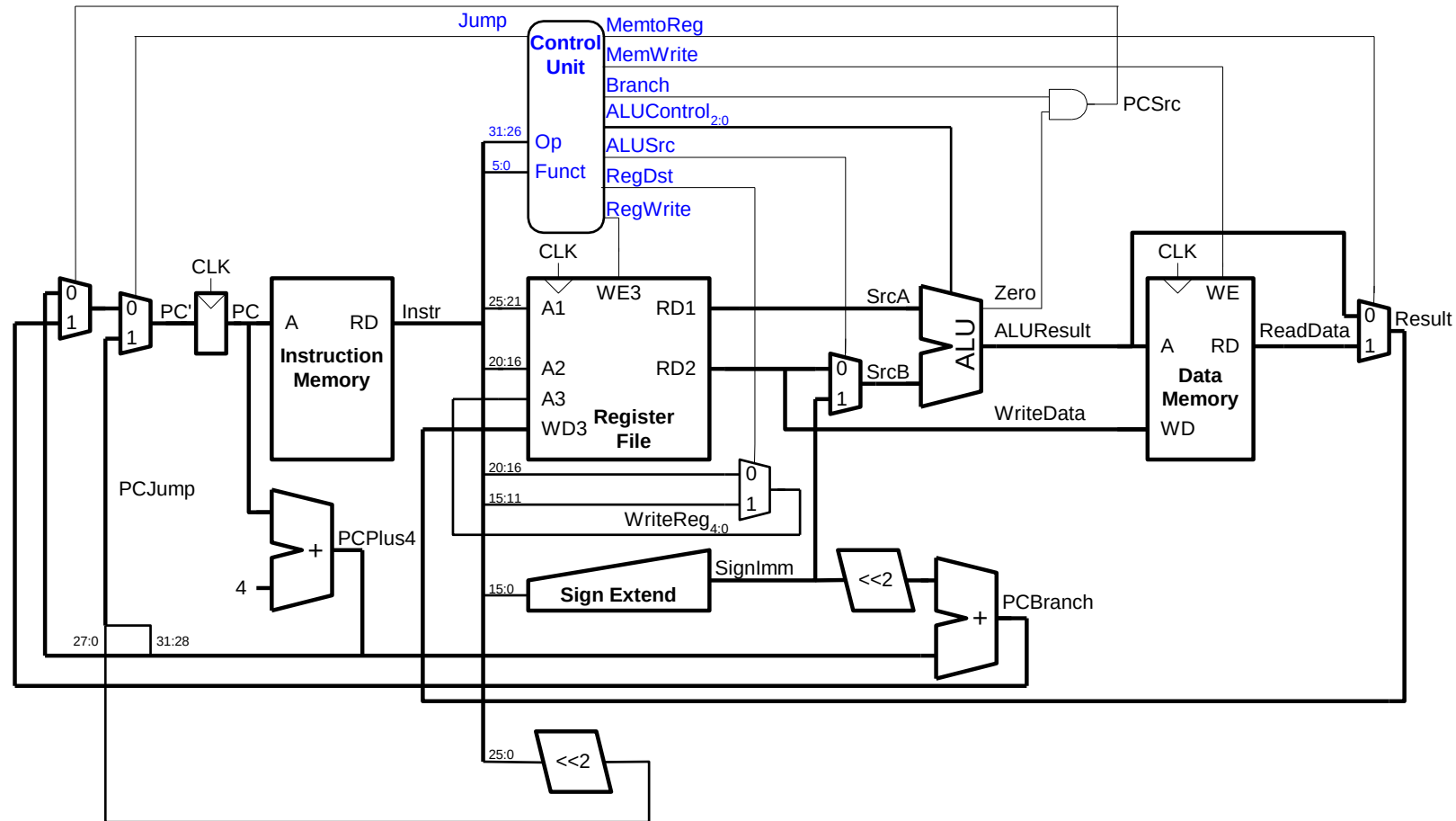


A Basic Multi-Cycle Microarchitecture

- Instruction processing cycle divided into “states”
 - A stage in the instruction processing cycle can take multiple states
- A multi-cycle microarchitecture sequences from state to state to process an instruction
 - The behavior of the machine in a state is completely determined by control signals in that state
- The behavior of the entire processor is specified fully by a *finite state machine*
- In a state (clock cycle), control signals control two things:
 - How the datapath should process the data
 - How to generate the control signals for the (next) clock cycle

One Example Multi-Cycle Microarchitecture

Remember: Single-Cycle MIPS Processor



Multi-cycle MIPS Processor

- Single-cycle microarchitecture:
 - cycle time limited by longest instruction (1w) → low clock frequency
 - three adders/ALUs and two memories → high hardware cost
- Multi-cycle microarchitecture:
 - + higher clock frequency
 - + simpler instructions run faster
 - + reuse expensive hardware across multiple cycles
 - sequencing overhead paid many times
 - hardware overhead for storing intermediate results
- Same design steps: datapath & control

What Do We Want To Optimize

- Single Cycle Architecture uses two memories
 - One memory stores instructions, the other data
 - We want to use a single memory (Smaller size)

What Do We Want To Optimize

- Single Cycle Architecture uses two memories
 - One memory stores instructions, the other data
 - We want to use a single memory (Smaller size)
- Single Cycle Architecture needs three adders
 - ALU, PC, Branch address calculation
 - We want to use the ALU for all operations (smaller size)

What Do We Want To Optimize

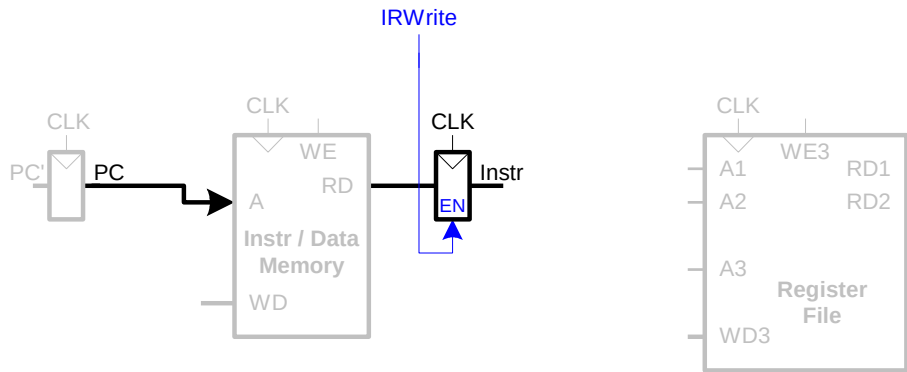
- Single Cycle Architecture uses two memories
 - One memory stores instructions, the other data
 - We want to use a single memory (Smaller size)
- Single Cycle Architecture needs three adders
 - ALU, PC, Branch address calculation
 - We want to use the ALU for all operations (smaller size)
- In Single Cycle Architecture all instructions take one cycle
 - The most complex operation slows down everything!
 - Divide all instructions into multiple steps
 - Simpler instructions can take fewer cycles (average case may be faster)

Consider the lw instruction

- For an instruction such as: `lw $t0, 0x20($t1)`
- We need to:
 - Read the instruction from memory
 - Then read **\$t1** from register array
 - Add the immediate value (**0x20**) to calculate the memory address
 - Read the content of this address
 - Write to the register **\$t0** this content

Multi-cycle Datapath: instruction fetch

- First consider executing lw
 - STEP 1: Fetch instruction

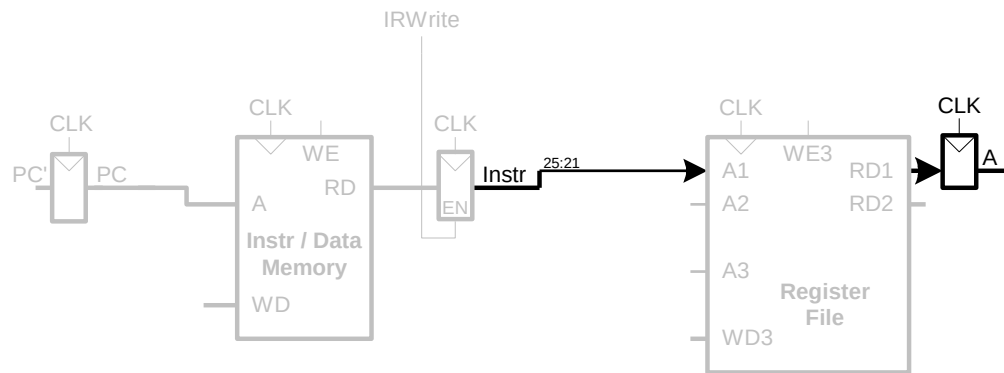


read from the memory location $[rs] + imm$ to location $[rt]$

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

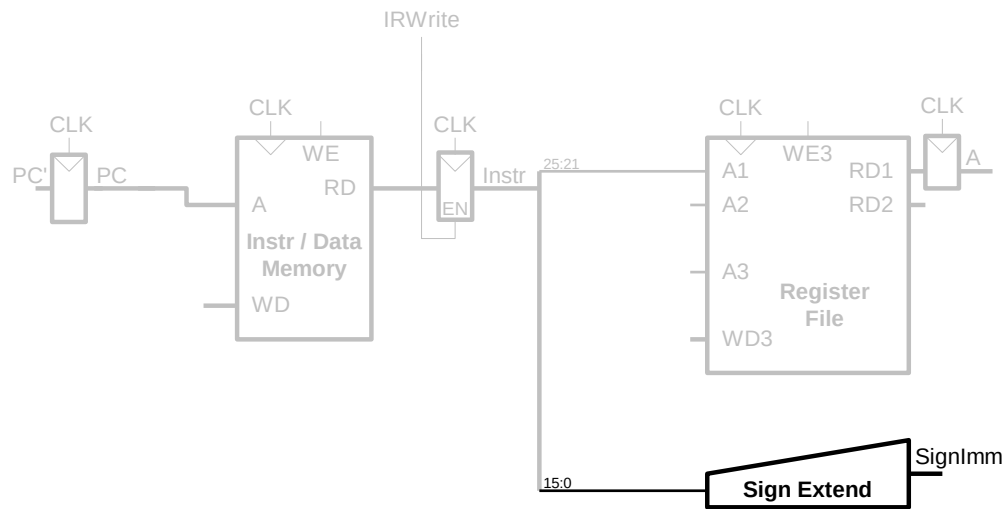
Multi-cycle Datapath: 1w register read



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

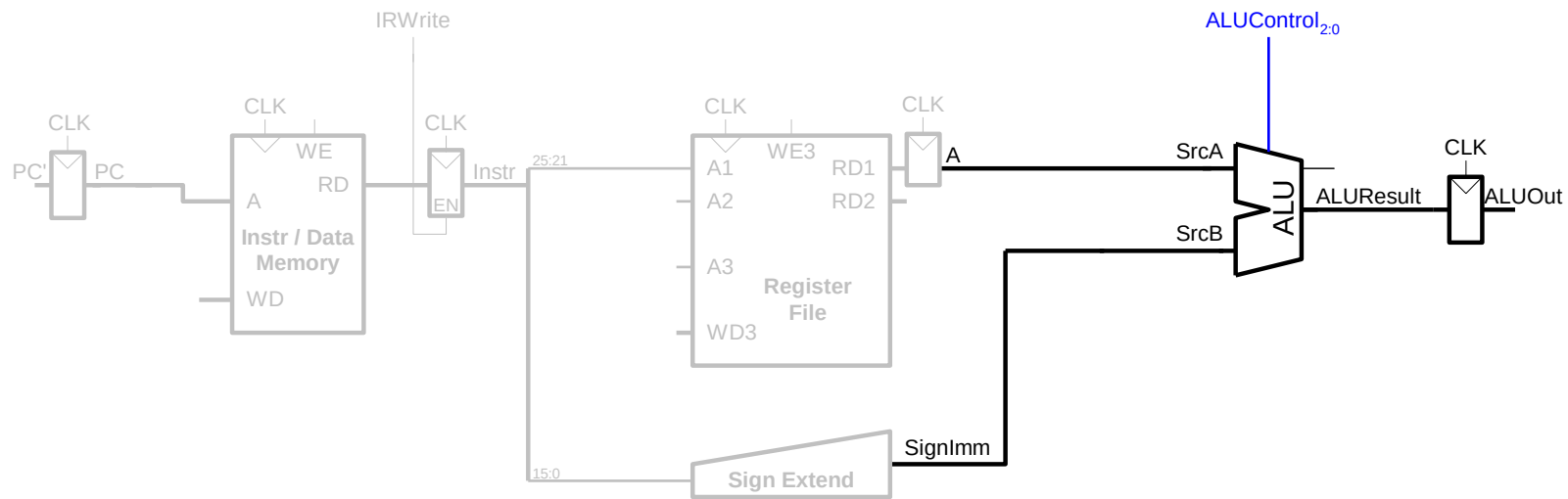
Multi-cycle Datapath: 1w immediate



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

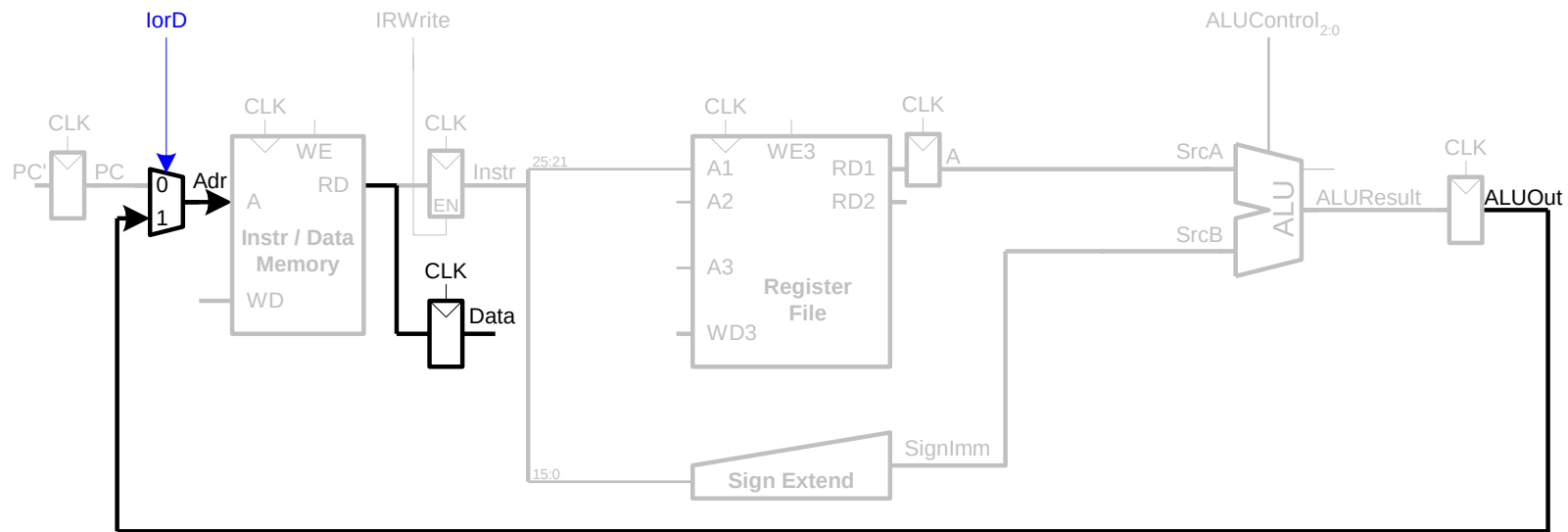
Multi-cycle Datapath: 1w address



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

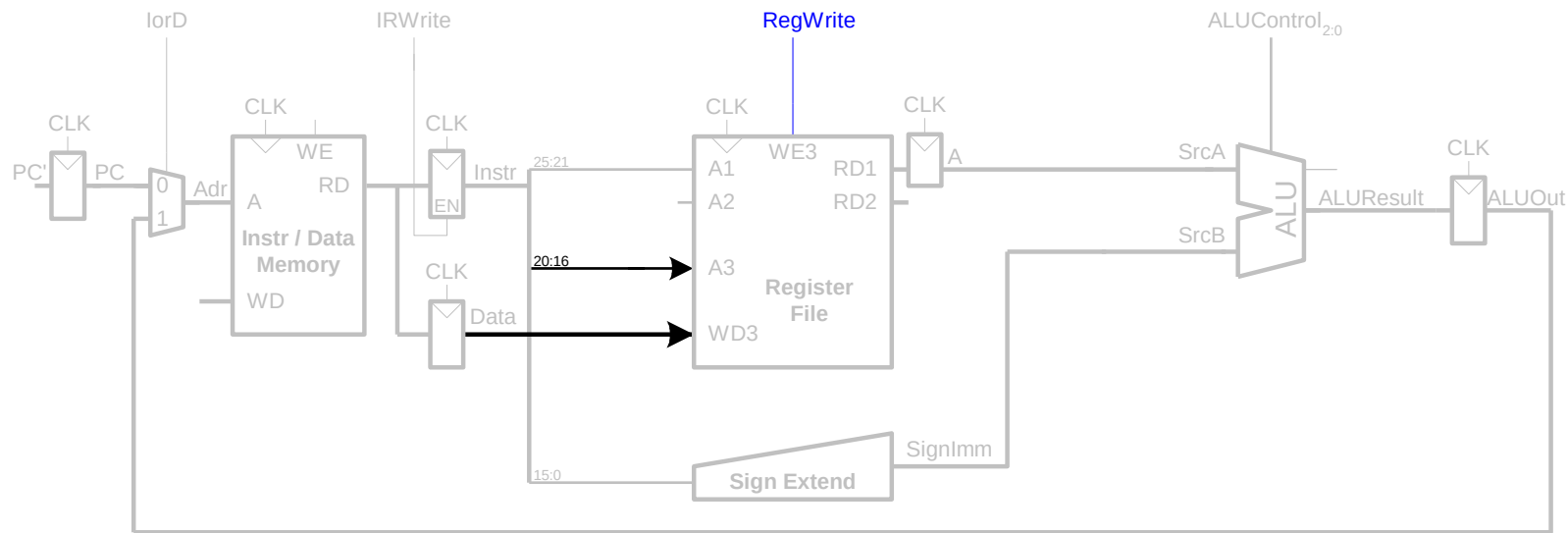
Multi-cycle Datapath: 1w memory read



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

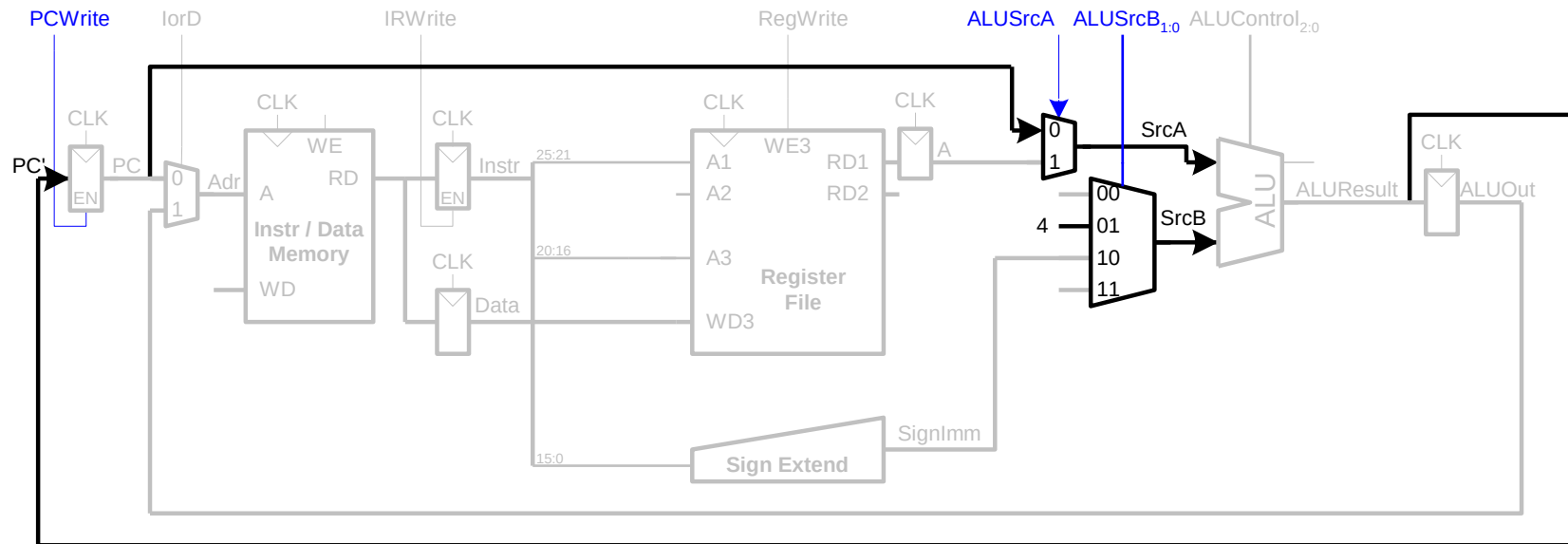
Multi-cycle Datapath: 1w write register



I-Type

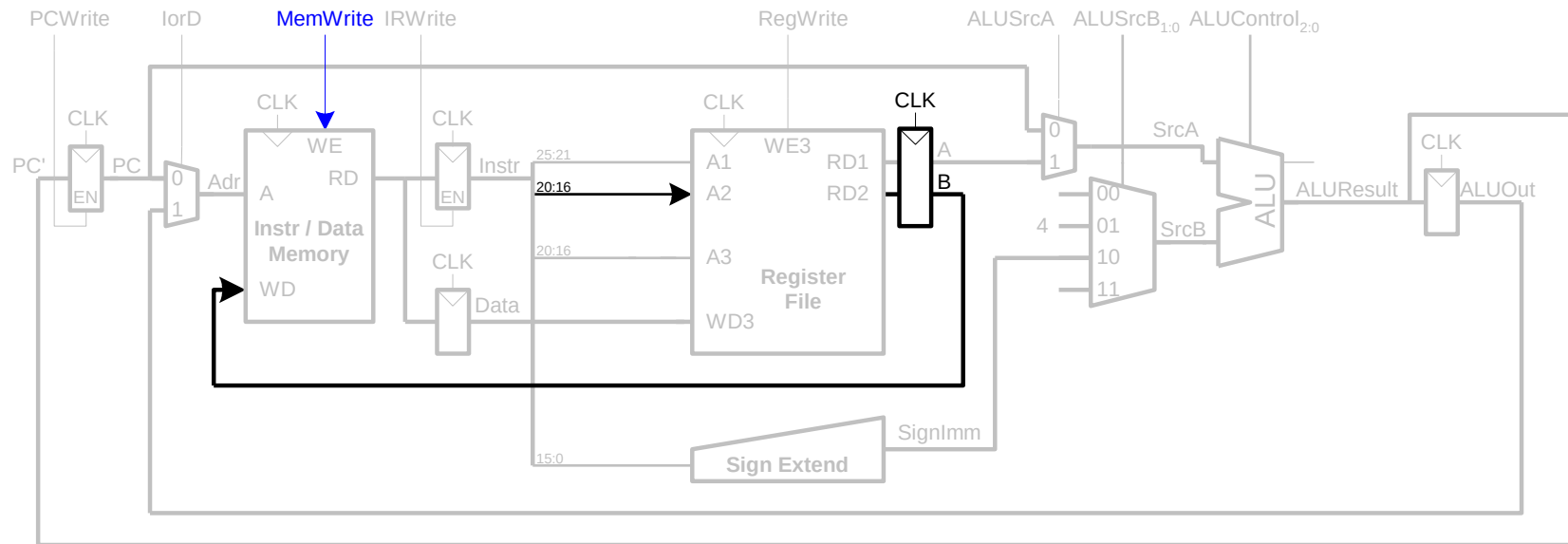
op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Multi-cycle Datapath: increment PC



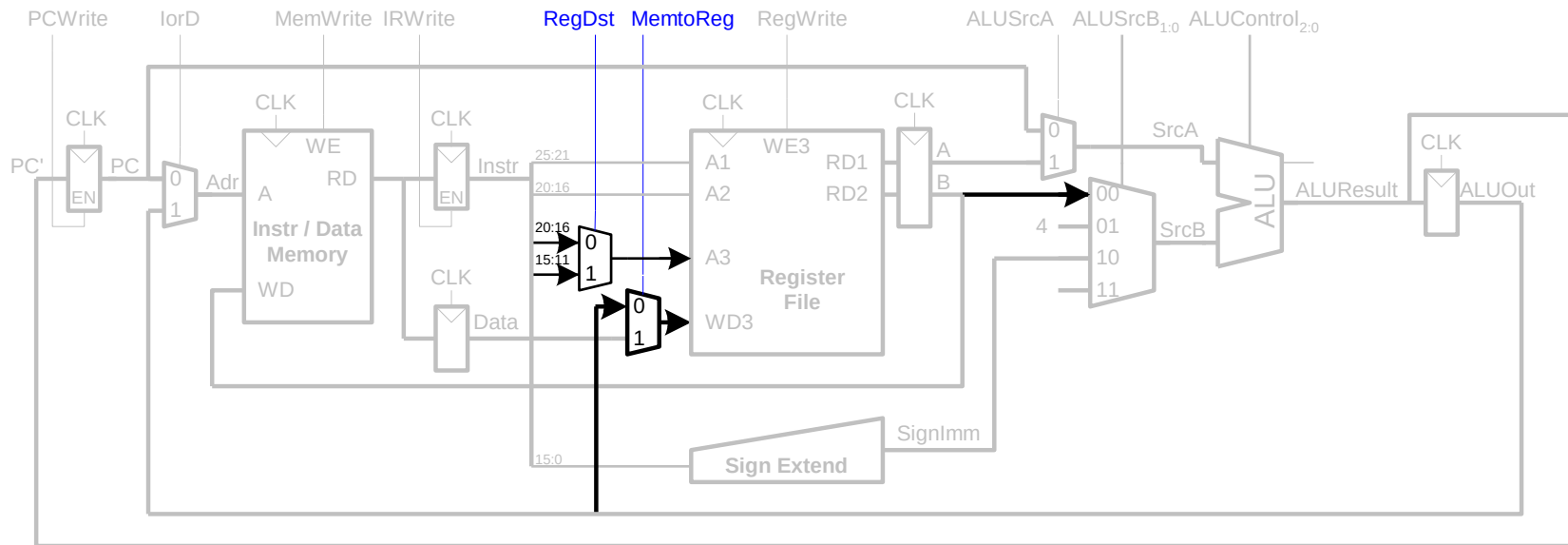
Multi-cycle Datapath: sw

- Write data in rt to memory



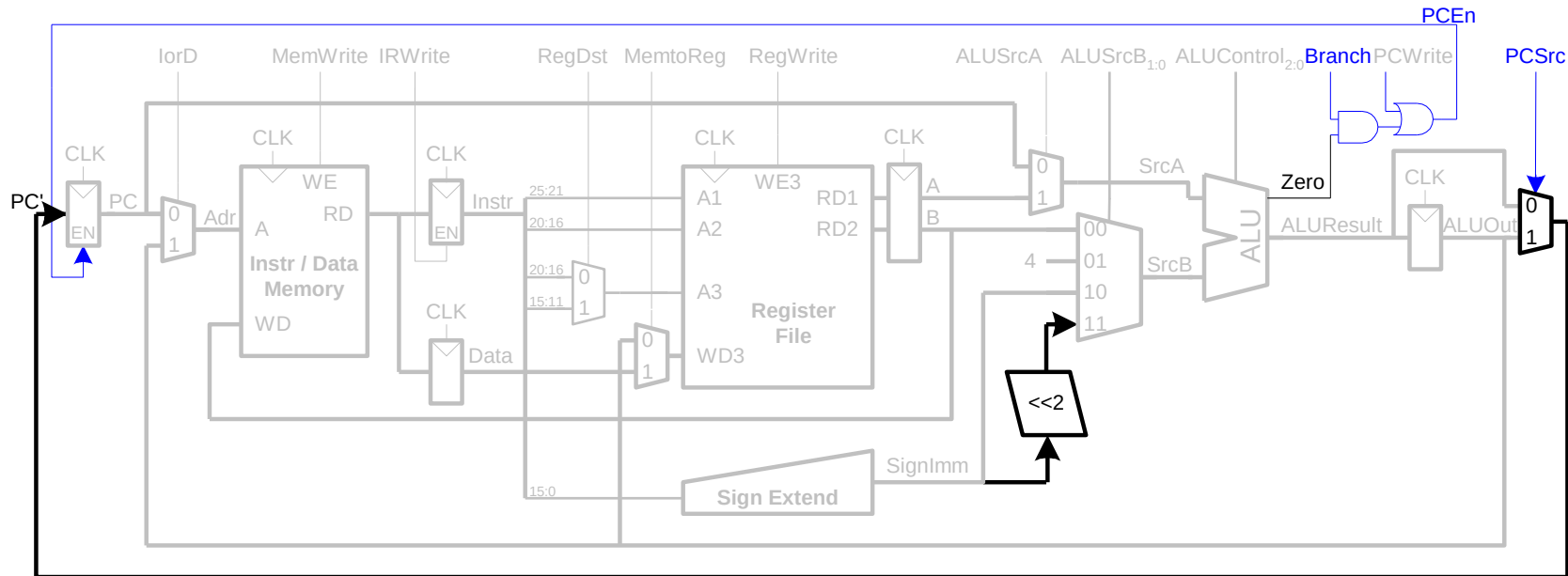
Multi-cycle Datapath: R-type Instructions

- Read from rs and rt
 - Write ALUResult to register file
 - Write to rd (instead of rt)

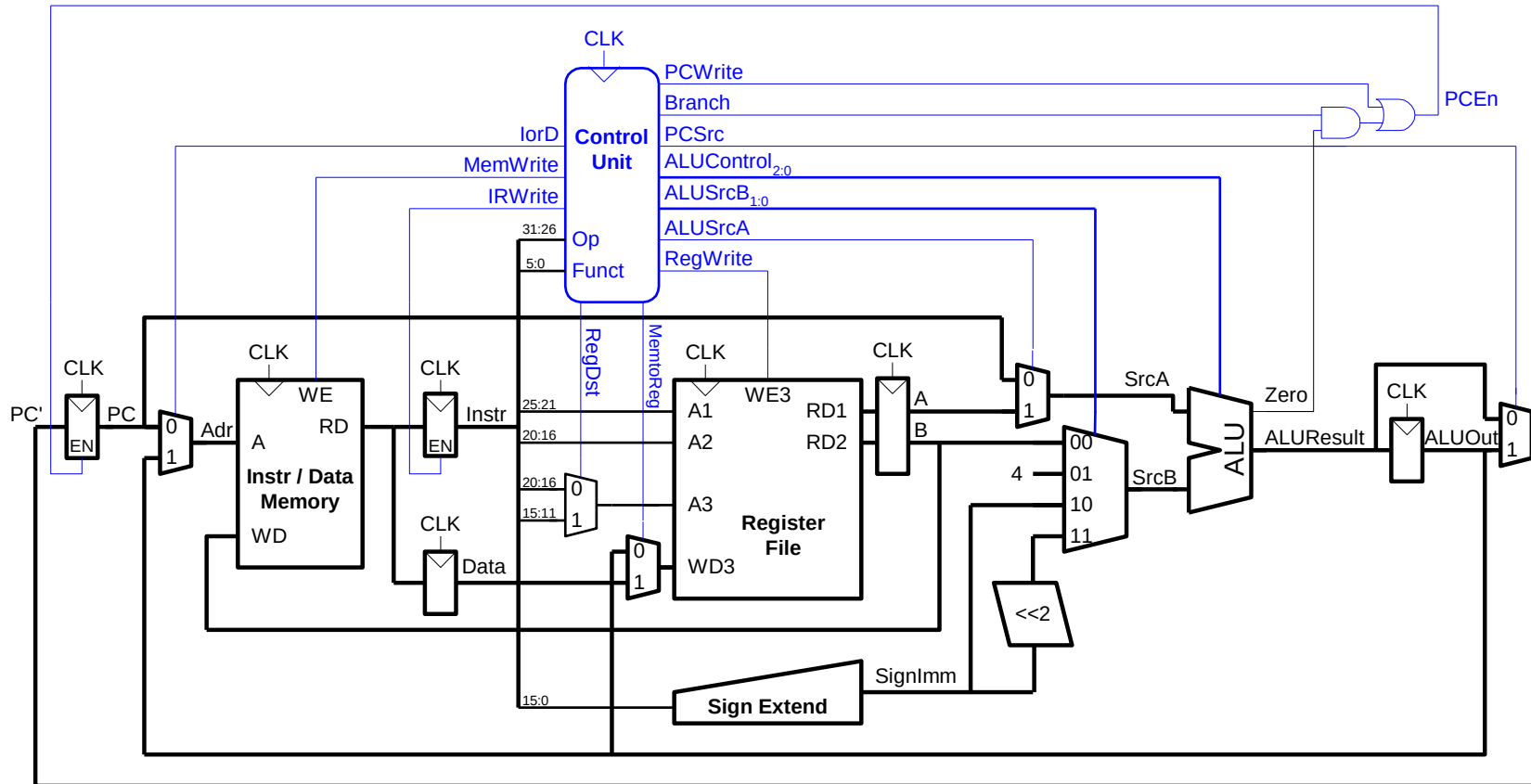


Multi-cycle Datapath: beq

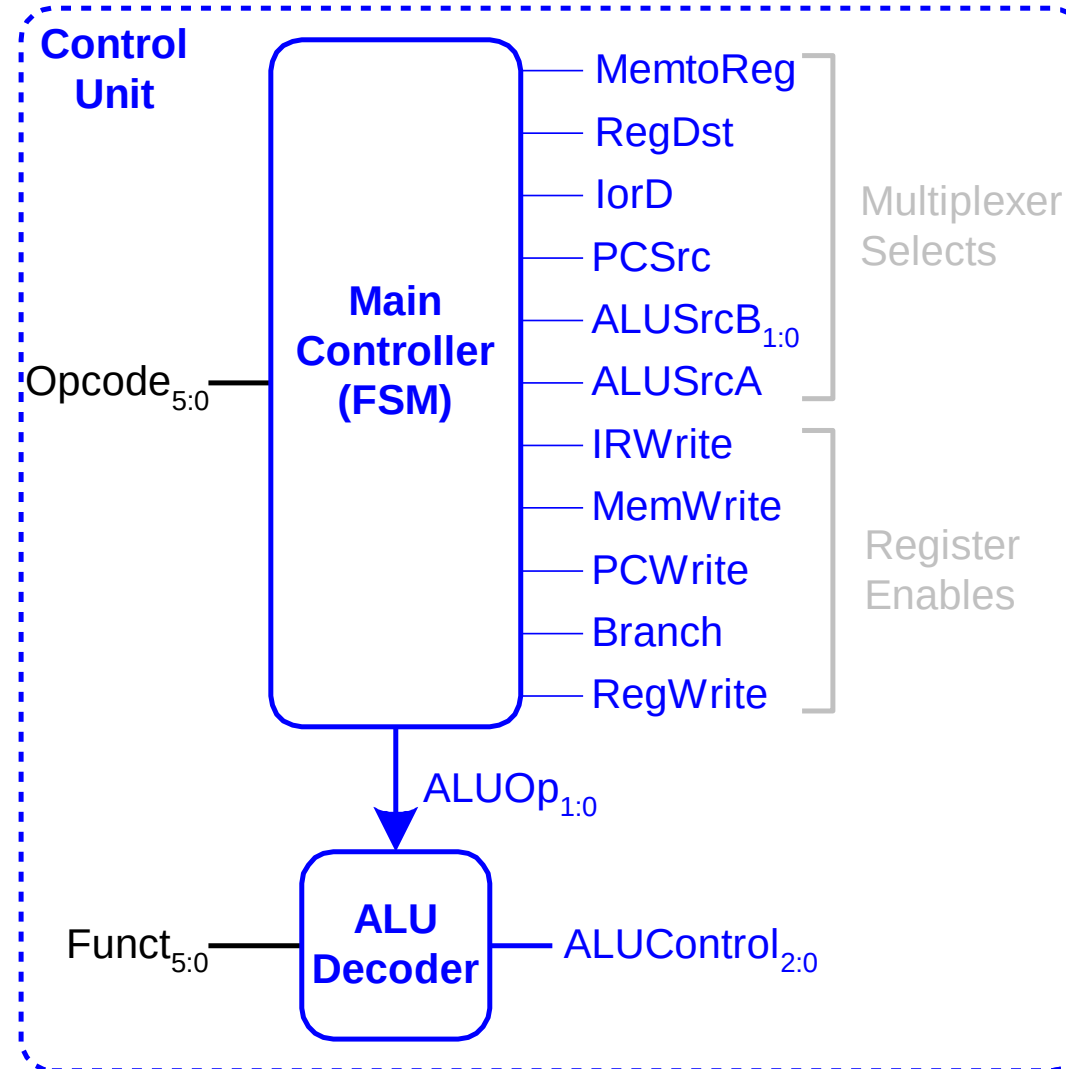
- Determine whether values in rs and rt are equal
 - Calculate branch target address:
 $BTA = (\text{sign-extended immediate} \ll 2) + (\text{PC} + 4)$



Complete Multi-cycle Processor

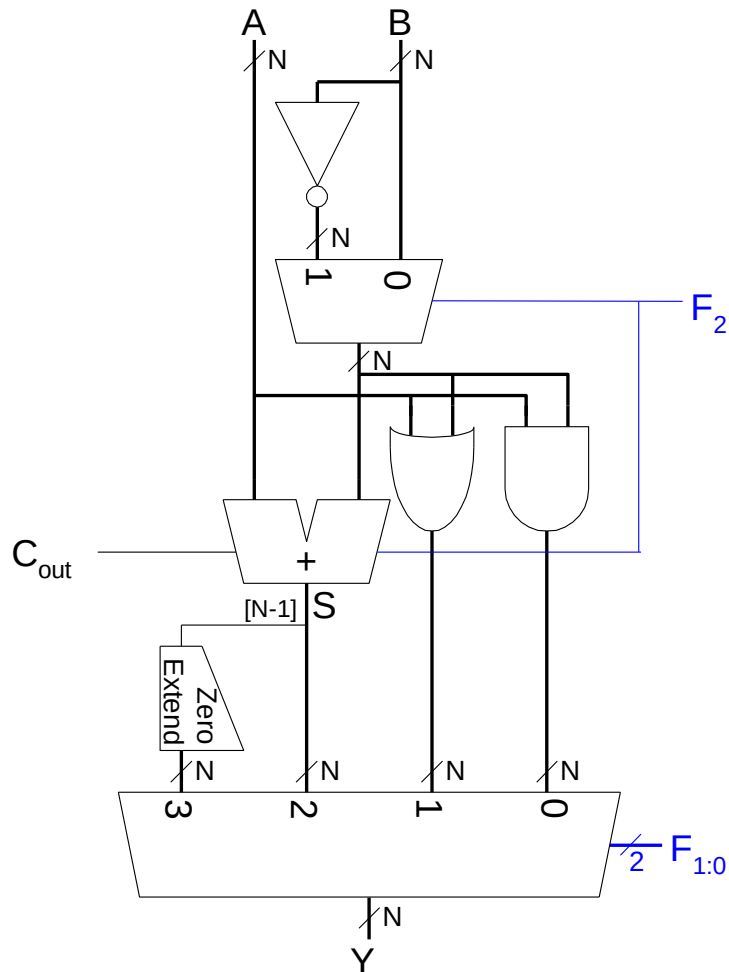


Control Unit



Example: ALU Design

- **ALU operation** ($F_{2:0}$) comes from the control logic



$F_{2:0}$	Function
000	A & B
001	A B
010	A + B
011	not used
100	A & $\sim$ B
101	A $\sim$ B
110	A - B
111	SLT